

第五章 运输层



福州大学
FUZHOU UNIVERSITY

1

第五章 运输层

知识点：运输层协议概述，用户数据报协议UDP，传输控制层协议TCP，可靠传输的工作原理，TCP报文段的首部格式，TCP可靠传输的实现，TCP的流量控制，TCP的拥塞控制，TCP的运输连接管理。

重点：用户数据报协议UDP，传输控制层协议TCP，可靠传输的工作原理，TCP报文段的首部格式，TCP可靠传输的实现，TCP的流量控制，TCP的拥塞控制。

难点：用户数据报协议UDP，传输控制层协议TCP，可靠传输的工作原理，TCP报文段的首部格式，TCP可靠传输的实现，TCP的流量控制，TCP的拥塞控制。



福州大学
FUZHOU UNIVERSITY

2

主要内容

计算机网络

5.1 运输层协议概述

5.1.1 进程之间的通信

5.1.2 因特网的运输层协议

5.1.3 运输层的复用与分用

5.2 用户数据报协议 UDP

5.2.1 UDP 概述

5.2.2 UDP 的首部格式



福州大学
FUZHOU UNIVERSITY

3

主要内容

计算机网络

5.3 传输控制协议 TCP

5.3.1 TCP 的主要特点

5.3.2 TCP 报文段结构

5.3.3 TCP 的可靠传输

5.3.4 TCP 的流量控制

5.3.5 TCP 的连接管理

5.4 拥塞控制

5.4.1 拥塞的原因与危害

5.4.2 拥塞控制的基本方法

5.4.3 TCP 的拥塞控制



福州大学
FUZHOU UNIVERSITY

4

5.1 运输层协议概述

计算机网络

5.1.1 进程之间的通信

5.1.2 因特网的运输层协议

5.1.3 运输层的复用与分用



福州大学
FUZHOU UNIVERSITY

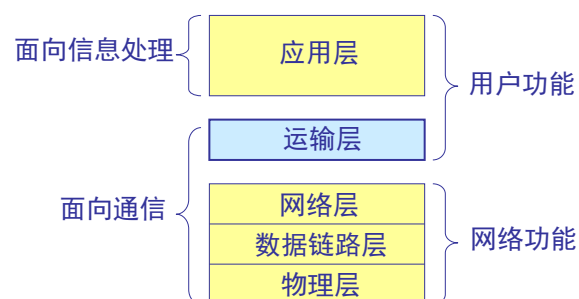
5

5.1 运输层协议概述

计算机网络

■ 5.1.1 进程之间的通信

- 从通信和信息处理的角度看，运输层向它上面的应用层提供通信服务，它属于面向通信部分的最高层，同时也是用户功能中的最低层。



福州大学
FUZHOU UNIVERSITY

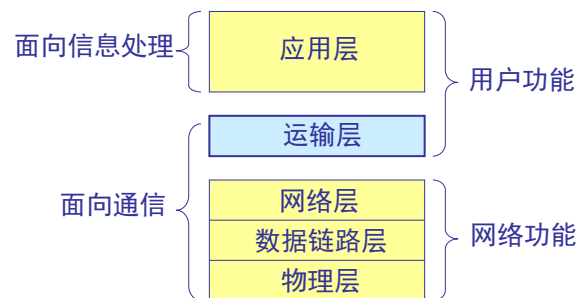
6

5.1 运输层协议概述

计算机网络

■ 5.1.1 进程之间的通信

- 当网络的边缘部分中的两个主机使用网络的核心部分的功能进行端到端的通信时，只有位于网络边缘部分的主机的协议栈才有运输层，而网络核心部分中的路由器在转发分组时都只用到下三层的功能。

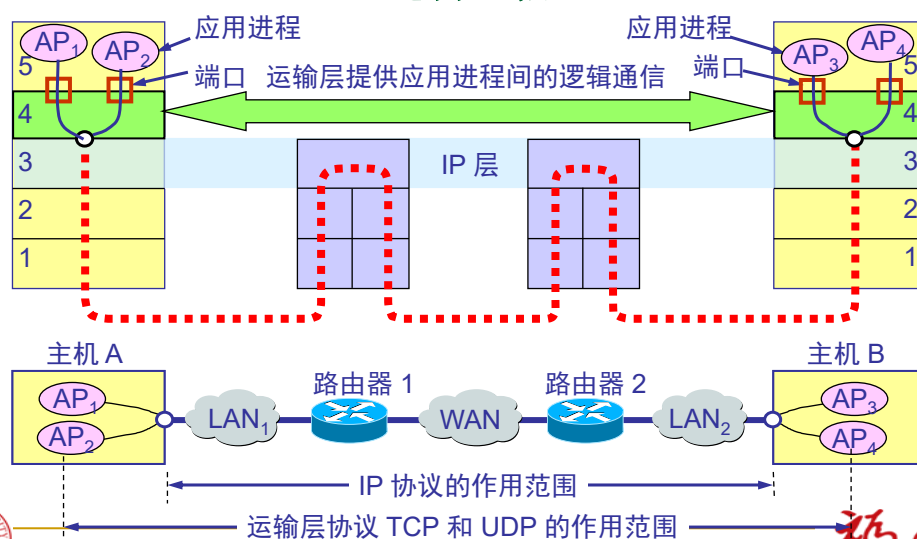


福州大学
FUZHOU UNIVERSITY

7

运输层为相互通信的应用进程提供了逻辑通信

计算机网络



福州大学
FUZHOU UNIVERSITY

8

5.1 运输层协议概述

计算机网络

■ 应用进程之间的通信

- 两个主机进行通信实际上就是两个主机中的**应用进程互相通信**。
- 应用进程之间的通信又称为**端到端的通信**。
- 运输层的一个很重要的功能就是复用和分用。应用层不同进程的报文通过不同的端口向下交到运输层，再往下就共用网络层提供的服务。
- “运输层提供应用进程间的**逻辑通信**”。“逻辑通信”的意思是：运输层之间的通信好像是沿水平方向传送数据。但事实上这两个运输层之间并没有一条水平方向的物理连接。



福州大学
FUZHOU UNIVERSITY

9

网络层和运输层有明显的区别

计算机网络

网络层是为主机之间提供逻辑通信，
而运输层为应用进程之间提供端到端的逻辑通信。



运输层协议和网络层协议的主要区别

福州大学
FUZHOU UNIVERSITY

10

运输层的作用

计算机网络

- 在一台主机中经常有**多个应用进程**同时分别和另一台主机中的多个应用进程通信。
- 这表明运输层有一个很重要的功能——**复用 (multiplexing)**和**分用 (demultiplexing)**。
- 根据应用程序的不同需求，运输层需要有两种不同的运输协议，即**面向连接的 TCP** 和**无连接的 UDP**。



福州大学
FUZHOU UNIVERSITY

11

屏蔽作用

计算机网络

- 运输层向高层用户**屏蔽**了下面网络核心的细节（如网络拓扑、所采用的路由选择协议等），它使应用进程看见的就是好像在两个运输层实体之间有一条**端到端的逻辑通信信道**。



福州大学
FUZHOU UNIVERSITY

12

两种不同的运输协议

计算机网络

- 但这条逻辑通信信道对上层的表现却因运输层使用的不同协议而有很大的差别。
- 当运输层采用面向连接的 **TCP** 协议时，尽管下面的网络是不可靠的（只提供尽最大努力服务），但这种逻辑通信信道就相当于一**条全双工的可靠信道**。
- 当运输层采用无连接的 **UDP** 协议时，这种逻辑通信信道是一条**不可靠信道**。

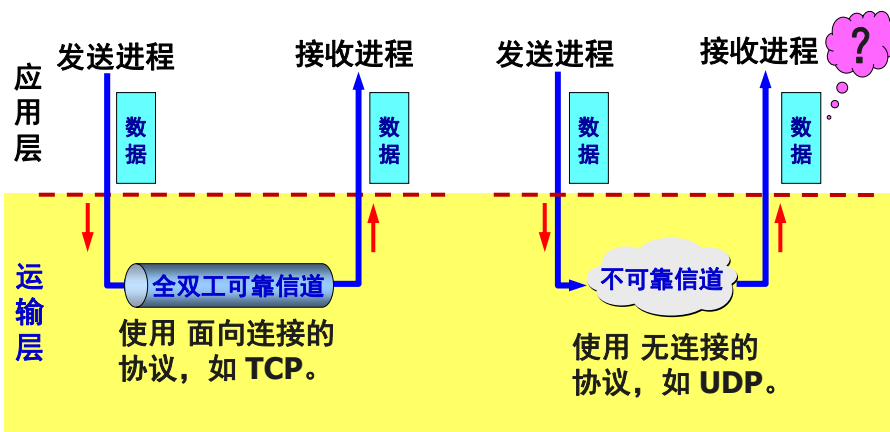


福州大学
FUZHOU UNIVERSITY

13

可靠信道与不可靠信道

计算机网络



福州大学
FUZHOU UNIVERSITY

14

运输层的主要功能

计算机网络

- 运输层为应用进程之间提供端到端的逻辑通信（但网络层是为主机之间提供逻辑通信）。
- 运输层还要对收到的报文进行差错检测。
- 运输层可选的功能：
 - 可靠数据传输、流量控制、拥塞控制



福州大学
FUZHOU UNIVERSITY

15

5.1.2 因特网的运输层协议

计算机网络

TCP/IP 的运输层有两个主要协议：

- (1) 用户数据报协议 **UDP** (User Datagram Protocol)
- (2) 传输控制协议 **TCP** (Transmission Control Protocol)



TCP/IP 体系中的运输层协议



福州大学
FUZHOU UNIVERSITY

16

TCP 与 UDP

计算机网络

- 两个对等运输实体在通信时传送的数据单位叫作**运输协议数据单元 TPDU (Transport Protocol Data Unit)**。
- TCP 传送的数据单位协议是 **TCP 报文段(segment)**。
- UDP 传送的数据单位协议是 **UDP 报文**或**用户数据报**。



福州大学
FUZHOU UNIVERSITY

17

TCP 与 UDP

计算机网络

- **UDP: 一种无连接协议**
 - 提供无连接服务。
 - 在传送数据之前不需要先建立连接。
 - 传送的数据单位协议是 **UDP 报文**或**用户数据报**。
 - 对方的运输层在收到 **UDP 报文**后，不需要给出任何确认。
 - 虽然 **UDP 不提供可靠交付**，但在某些情况下 **UDP** 是一种最有效的工作方式。



福州大学
FUZHOU UNIVERSITY

18

TCP 与 UDP

计算机网络

■ TCP：一种面向连接的协议

- 提供面向连接的服务。
- 传送的数据单位协议是 **TCP 报文段 (segment)**。
- **TCP 不提供广播或多播服务。**
- 由于 **TCP 要提供可靠的、面向连接的运输服务**，因此不可避免地增加了许多的开销。这不仅使协议数据单元的首部增大很多，还要占用许多的处理机资源。



福州大学
FUZHOU UNIVERSITY

19

还要强调两点

计算机网络

- 运输层的 **UDP** 用户数据报与网际层的**IP**数据报有很大区别。
 - **IP** 数据报要经过互连网中许多路由器的存储转发。
 - **UDP** 用户数据报是在运输层的端到端抽象的逻辑信道中传送的。
- **TCP** 报文段是在运输层抽象的端到端逻辑信道中传送，这种信道是可靠的全双工信道。但这样的信道却不知道究竟经过了哪些路由器，而这些路由器也根本不知道上面的运输层是否建立了 **TCP** 连接。



福州大学
FUZHOU UNIVERSITY

20

5.1.3 运输层的复用与分用

计算机网络

- **复用**是指在发送方不同的应用进程都可以使用同一个运输层协议传送数据（当然需要加上适当的首部）；
- **分用**是指接收方的运输层在剥去报文的首部后能够把这些数据正确交付到目的应用进程。
- 要能正确地将数据交付给指定应用进程，就必须给每个应用进程赋予一个明确的标志。
- 在**TCP/IP**网络中，使用一种与操作系统无关的协议端口号（**protocol port number**）（简称端口号）来实现对通信的应用进程的标志。

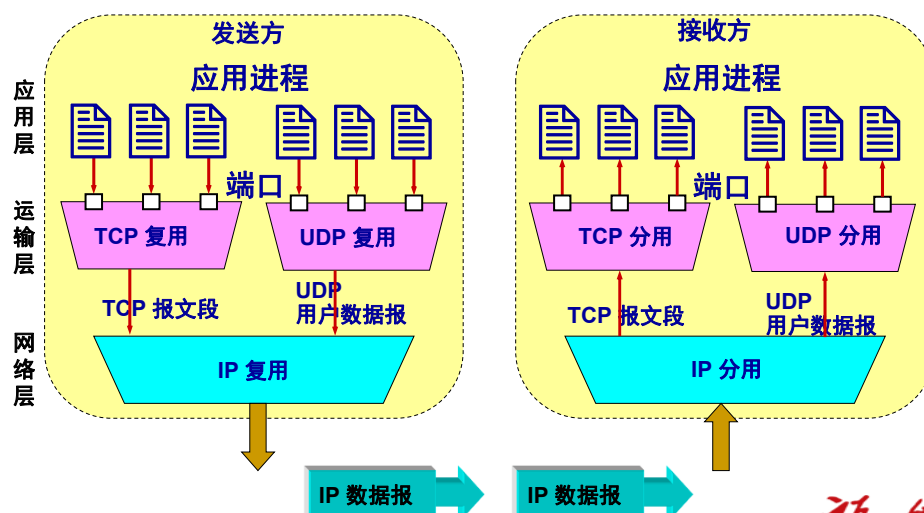


福州大学
FUZHOU UNIVERSITY

21

基于端口的复用和分用功能

计算机网络



福州大学
FUZHOU UNIVERSITY

22

5.1.3 运输层的复用与分用

计算机网络

- 运行在计算机中的进程是用进程标识符来标志的。
- 但运行在应用层的各种应用进程却不应当让计算机操作系统指派它的进程标识符。这是因为在互联网上使用的计算机的操作系统种类很多，而不同的操作系统又使用不同格式的进程标识符。
- 为了使运行不同操作系统的计算机的应用进程能够互相通信，就必须用统一的方法对 TCP/IP 体系的应用进程进行标志。



福州大学
FUZHOU UNIVERSITY

23

需要解决的问题

计算机网络

- 由于进程的创建和撤销都是动态的，发送方几乎无法识别其他机器上的进程。
- 有时我们会改换接收报文的进程，但并不需要通知所有发送方。
- 我们往往需要利用目的主机提供的功能来识别终点，而不需要知道实现这个功能的进程。



福州大学
FUZHOU UNIVERSITY

24

端口号 (protocol port number)

计算机网络

- 解决这个问题的方法就是在运输层使用**协议端口号 (protocol port number)**，或通常简称为**端口 (port)**。
- 虽然通信的终点是应用进程，但我们可以把端口想象是通信的终点，因为我们只要把要传送的报文交到目的主机的某一个合适的目的端口，剩下的工作（即最后交付目的进程）就由 **TCP** 来完成。



福州大学
FUZHOU UNIVERSITY

25

软件端口与硬件端口

计算机网络

- 两个不同的概念。
- 在协议栈层间的抽象的协议端口是**软件端口**。
- 路由器或交换机上的端口是**硬件端口**。
- 硬件端口是不同硬件设备进行交互的接口，而软件端口是应用层的各种协议进程与运输实体进行层间交互的一种地址。



福州大学
FUZHOU UNIVERSITY

26

TCP/IP 运输层端口

计算机网络

- 端口用一个 **16** 位端口号进行标志。
- 端口号只具有**本地意义**，即端口号只是为了标志**本计算机应用层中的各进程**。
- 在互联网中，不同计算机的相同端口号是没有联系的。

由此可见，两个计算机中的进程要互相通信，不仅必须知道对方的 IP 地址（为了找到对方的计算机），而且还要知道对方的端口号（为了找到对方计算机中的应用进程）。



福州大学
FUZHOU UNIVERSITY

27

两大类端口

计算机网络

(1) 服务器端使用的端口号

- 熟知端口，数值一般为 **0~1023**。
- 登记端口号，数值为 **1024~49151**，为没有熟知端口号的应用程序使用的。使用这个范围的端口号必须在 IANA 登记，以防止重复。

(2) 客户端使用的端口号

- 又称为**短暂端口号**，数值为 **49152~65535**，留给客户进程选择暂时使用。
- 当服务器进程收到客户进程的报文时，就知道了客户进程所使用的动态端口号。通信结束后，这个端口号可供其他客户进程以后使用。

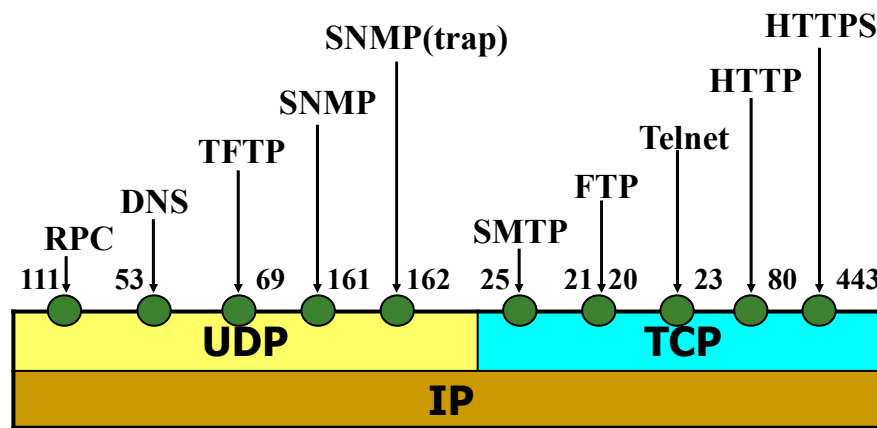


福州大学
FUZHOU UNIVERSITY

28

常用的熟知端口

计算机网络



福州大学
FUZHOU UNIVERSITY

29

5.2 用户数据报协议 UDP

计算机网络

- 5.2.1 UDP 概述
- 5.2.2 UDP 的首部格式



福州大学
FUZHOU UNIVERSITY

30

5.2.1 UDP概述

计算机网络

- **UDP** 只在 **IP** 的数据报服务之上增加了很少一点的功能：
 - 复用和分用的功能
 - 差错检测的功能
- 虽然 **UDP** 用户数据报只能提供不可靠的交付，但 **UDP** 在某些方面有其特殊的优点。
 - 发送数据之前不需要建立连接
 - **UDP** 的主机不需要维持复杂的连接状态表。
 - **UDP** 用户数据报只有 8 字节的首部开销。
 - 网络出现的拥塞不会使源主机的发送速率降低。这对某些实时应用是很重要的。



福州大学
FUZHOU UNIVERSITY

31

UDP 的主要特点

计算机网络

- **(1) UDP 是无连接的**，发送数据之前不需要建立连接，因此减少了开销和发送数据之前的时延。
- **(2) UDP 使用尽最大努力交付**，即不保证可靠交付，因此主机不需要维持复杂的连接状态表。
- **(3) UDP 是面向报文的**。**UDP** 对应用层交下来的报文，既合并，也不拆分，而是保留这些报文的边界。**UDP** 一次交付一个完整的报文。
- **(4) UDP 没有拥塞控制**，因此网络出现的拥塞不会使源主机的发送速率降低。这对某些实时应用是很重要的。很适合多媒体通信的要求。



福州大学
FUZHOU UNIVERSITY

32

UDP 的主要特点

计算机网络

- (5) UDP 支持一对一、一对多、多对一和多对多的交互通信。
- (6) UDP 的首部开销小，只有 8 个字节，比 TCP 的 20 个字节的首部要短。



福州大学
FUZHOU UNIVERSITY

33

面向报文的 UDP

计算机网络

- 发送方 UDP 对应用程序交下来的报文，在添加首部后就向下交付 IP 层。UDP 对应用层交下来的报文，既不合并，也不拆分，而是保留这些报文的边界。
- 应用层交给 UDP 多长的报文，UDP 就照样发送，即一次发送一个报文。



福州大学
FUZHOU UNIVERSITY

34

面向报文的 UDP

计算机网络

- 接收方 **UDP** 对 **IP** 层交上来的 **UDP** 用户数据报，在去除首部后就原封不动地交付上层的应用进程，一次交付一个完整的报文。
- 应用程序必须选择合适大小的报文。
 - 若报文太长，UDP 把它交给 IP 层后，IP 层在传送时可能要进行分片，这会降低 IP 层的效率。
 - 若报文太短，UDP 把它交给 IP 层后，会使 IP 数据报的首部的相对长度太大，这也降低了 IP 层的效率。

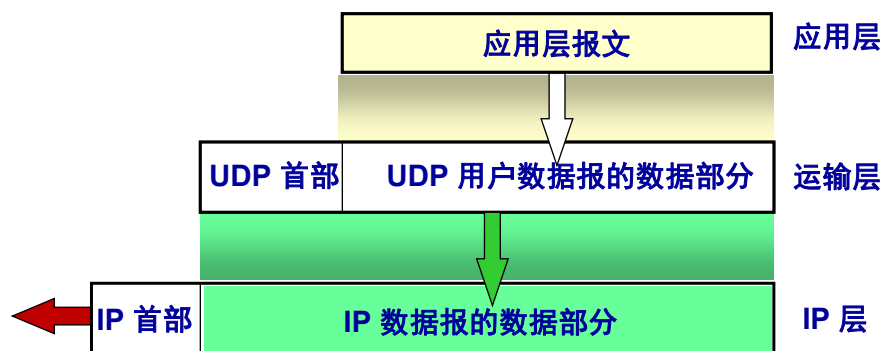


福州大学
FUZHOU UNIVERSITY

35

UDP 是面向报文的

计算机网络



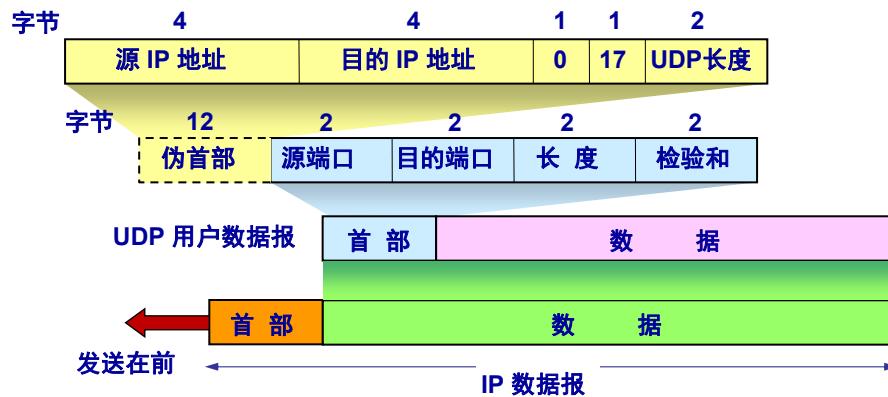
福州大学
FUZHOU UNIVERSITY

36

5.2.2 UDP 的首部格式

计算机网络

用户数据报 UDP 有两个字段：数据字段和首部字段。
首部字段很简单，只有 8 个字节。



发送在前

IP 数据报

UDP 用户数据报的首部和伪首部



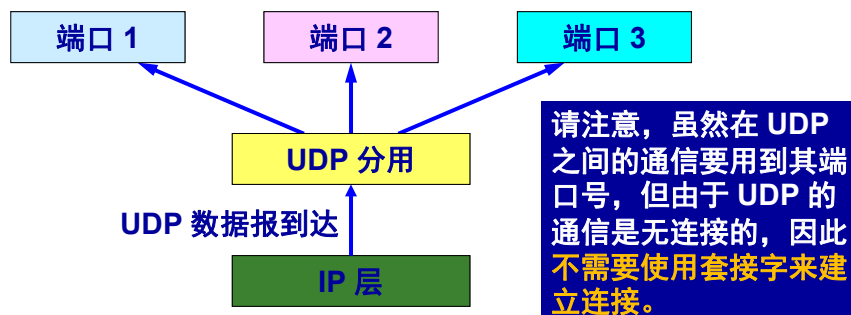
福州大学
FUZHOU UNIVERSITY

37

UDP 基于端口的分用

计算机网络

当运输层从 IP 层收到 UDP 数据报时，就根据首部中的目的端口，把 UDP 数据报通过相应的端口，上交最后的终点——应用进程。

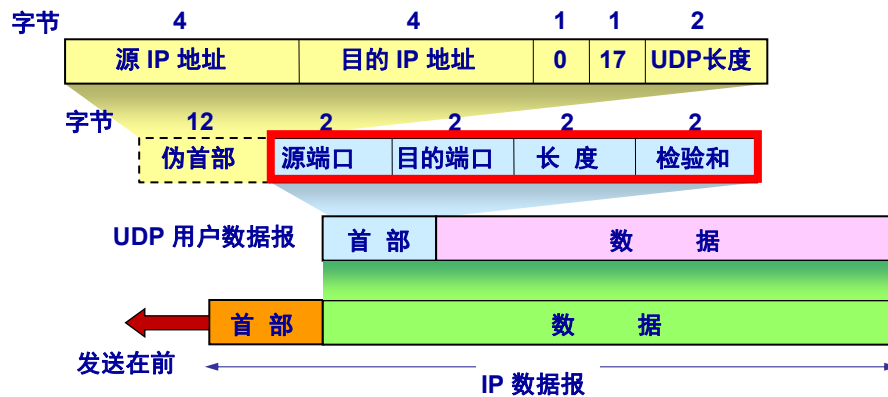


福州大学
FUZHOU UNIVERSITY

38

用户数据报 UDP 有两个字段：数据字段和首部字段。首部字段有 8 个字节，由 4 个字段组成，每个字段都是 2 个字节。

计算机网络

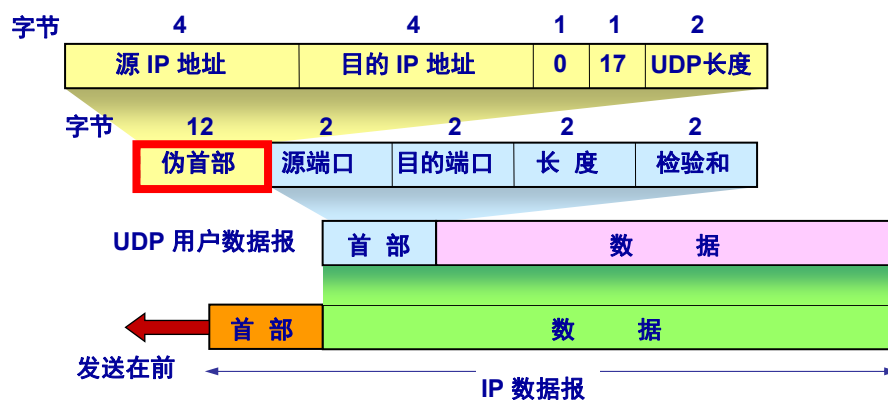


福州大学
FUZHOU UNIVERSITY

39

在计算检验和时，临时把“伪首部”和 UDP 用户数据报连接在一起。伪首部仅仅是为了计算检验和。

计算机网络



福州大学
FUZHOU UNIVERSITY

40

计算 UDP 检验和的例子

计算机网络

12 字节 伪首部	153.19.8.104				10011001 00010011 → 153.19
	171.3.14.11				00001000 01101000 → 8.104
	全 0	17	15		10101011 00000011 → 171.3
8 字节 UDP 首部	1087		13		00001110 00001011 → 14.11
	15		全 0		00000000 00010001 → 0 和 17
7 字节 数据	数据	数据	数据	数据	00000000 00001111 → 15
	数据	数据	数据	全 0	00000100 00111111 → 1087
					00000000 00001101 → 13
					00000000 00001111 → 15
					00000000 00000000 → 0 (检验和)
					01010100 01000101 → 数据
					01010011 01010100 → 数据
					01001001 01001110 → 数据
					01000111 00000000 → 数据和 0 (填充)
					10010110 11101101 → 求和得出的结果
					01101001 00010010 → 检验和

UDP的检验和是把首部
和数据部分一起都
检验。

填充

按二进制反码运算求和
将得出的结果求反码



福州大学
FUZHOU UNIVERSITY

41

5.3 传输控制协议 TCP

计算机网络

- 5.3.1 TCP 最主要的特点
- 5.3.2 TCP 报文段结构
- 5.3.3 TCP 的可靠传输
- 5.3.4 TCP 的流量控制
- 5.3.5 TCP 的连接管理



福州大学
FUZHOU UNIVERSITY

42

5.3.1 TCP 最主要的特点

计算机网络

- TCP 是面向连接的运输层协议。
- 每一条 TCP 连接只能有两个端点 (endpoint)，每一条 TCP 连接只能是点对点的（一对一）。
- TCP 提供可靠交付的服务。
- TCP 提供全双工通信。
- 面向字节流
 - TCP 中的“流” (stream)指的是流入或流出进程的字节序列。
 - “面向字节流”的含义是：虽然应用程序和 TCP 的交互是一次一个数据块，但 TCP 把应用程序交下来的数据看成仅仅是一连串无结构的字节流。



福州大学
FUZHOU UNIVERSITY

43

TCP 面向流的概念

计算机网络

- TCP 不保证接收方应用程序所收到的数据块和发送方应用程序所发出的数据块具有对应大小的关系。
- 但接收方应用程序收到的字节流必须和发送方应用程序发出的字节流完全一样。



福州大学
FUZHOU UNIVERSITY

44

注意

计算机网络

- **TCP** 连接是一条**虚连接**而不是一条真正的物理连接。
- **TCP** 对应用进程一次把多长的报文发送到**TCP** 的缓存中是不关心的。
- **TCP** 根据对方给出的**窗口值**和**当前网络拥塞**的程度来决定一个报文段应包含多少个字节（**UDP** 发送的报文长度是应用进程给出的）。
- **TCP** 可把太长的数据块划分短一些再传送。
- **TCP** 也可等待积累有足够多的字节后再构成报文段发送出去。



福州大学
FUZHOU UNIVERSITY

47

TCP 连接的任何一方都能够发送和接收数据

计算机网络

- 通信是全双工方式。
- 发送方的应用进程按照自己产生数据的规律，不断地把数据块陆续写入到 **TCP** 的发送缓存中。**TCP** 再从发送缓存中取出一定数量的数据，将其组成 **TCP 报文段(segment)**逐个传送给 **IP** 层，然后发送出去。
- 接收方从 **IP** 层收到 **TCP** 报文段后，先把它暂存在接收缓存中，然后让接收方的应用进程从接收缓存中将数据块逐个读取。



福州大学
FUZHOU UNIVERSITY

48

一对一的通信

计算机网络

- 由于运输层的通信是面向连接的，因此**TCP** 每一条连接上的通信只能是一对一的，而不可能是一对多、多对一或多对多的。



福州大学
FUZHOU UNIVERSITY

49

TCP 的连接

计算机网络

- **TCP** 把连接作为最基本的抽象。
- 每一条 **TCP** 连接有两个端点。
- **TCP** 连接的端点不是主机，不是主机的**IP** 地址，不是应用进程，也不是运输层的协议端口。**TCP** 连接的端点叫做套接字 (**socket**) 或插口。
- 端口号拼接到 (**contatenated with**) **IP** 地址即构成了套接字。



福州大学
FUZHOU UNIVERSITY

50

套接字 (socket)

计算机网络

套接字 socket = (IP地址 : 端口号) (5-1)

每一条 TCP 连接**唯一**地被通信两端的**两个端点**
(即两个套接字) 所确定。即:

TCP 连接 ::= {socket1, socket2}
= {(IP1: port1), (IP2: port2)} (5-2)



福州大学
FUZHOU UNIVERSITY

51

TCP 连接, IP 地址, 套接字

计算机网络

- TCP 连接就是由协议软件所提供的一种抽象。
- TCP 连接的端点是个很抽象的套接字, 即 (IP 地址: 端口号)。
- 同一个 IP 地址可以有多个不同的 TCP 连接。
- 同一个端口号也可以出现在多个不同的 TCP 连接中。



福州大学
FUZHOU UNIVERSITY

52

Socket 有多种不同的意思

计算机网络

- 应用编程接口 **API** 称为 **socket API**, 简称为 **socket**。
- **socket API** 中使用的一个函数名也叫作 **socket**。
- 调用 **socket** 函数的端点称为 **socket**。
- 调用 **socket** 函数时其返回值称为 **socket** 描述符, 可简称为 **socket**。
- 在操作系统内核中连网协议的 **Berkeley** 实现, 称为 **socket** 实现。

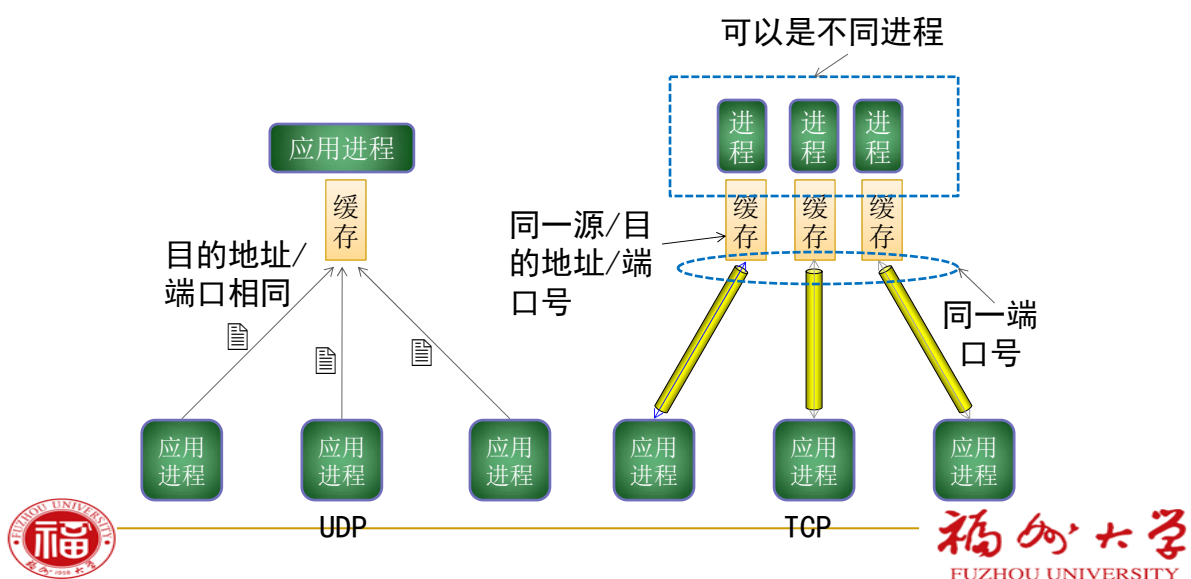


福州大学
FUZHOU UNIVERSITY

53

UDP与TCP复用的区别

计算机网络



54

可靠传输

计算机网络

- 理想的传输条件有以下**两个特点**:
 - (1) 传输信道不产生差错。
 - (2) 不管发送方以多快的速度发送数据，接收方总是来得及处理收到的数据。
- 在这样的理想传输条件下，不需要采取任何措施就能够实现可靠传输。
- **然而实际的网络都不具备以上两个理想条件。**必须使用一些可靠传输协议，在不可靠的传输信道实现可靠传输。



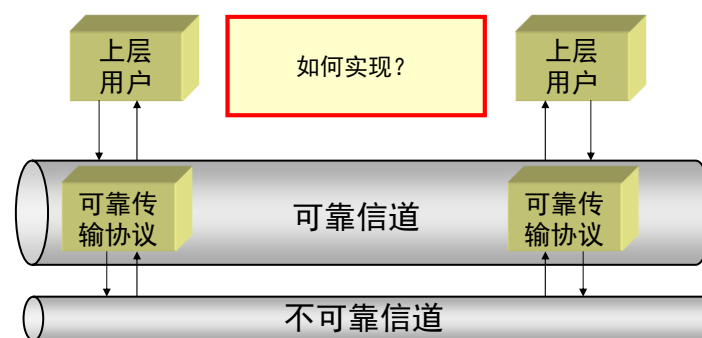
福州大学
FUZHOU UNIVERSITY

55

可靠传输

计算机网络

- 在不可靠的信道上实现可靠的数据传输
- 为上层提供一条可靠的逻辑通道



福州大学
FUZHOU UNIVERSITY

56

停止等待协议

计算机网络

- “停止等待”就是每发送完一个分组就停止发送，等待对方的确认。在收到确认后再发送下一个分组。
- 全双工通信的双方既是发送方也是接收方。
- 为了讨论问题的方便，我们仅考虑 **A** 发送数据而 **B** 接收数据并发送确认。因此 **A** 叫做**发送方**，而 **B** 叫做**接收方**。

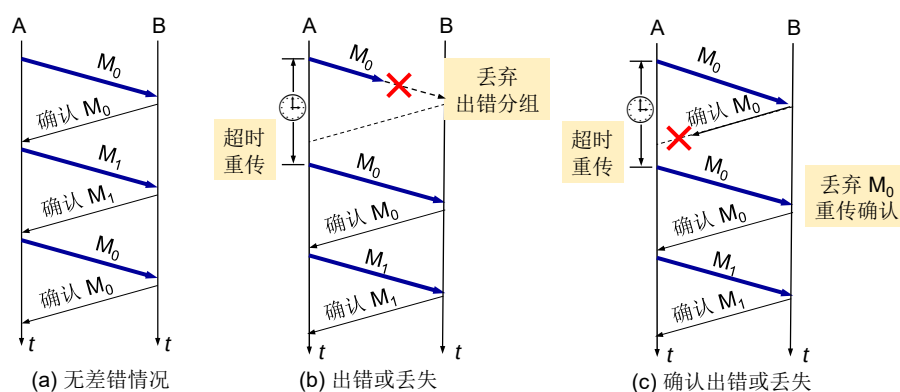


福州大学
FUZHOU UNIVERSITY

57

停止等待协议

计算机网络



A 发送分组 M_1 ，发完就暂停发送，等待 B 的确认 (ACK)。B 收到了 M_1 向 A 发送 ACK。A 在收到了对 M_1 的确认后，就再发送下一个分组 M_2 。



福州大学
FUZHOU UNIVERSITY

58

请注意

计算机网络

- 在发送完一个分组后，必须**暂时保留**已发送的分组的副本，以备重发。
- **分组和确认分组都必须进行编号。**
- 超时计时器的重传时间应当比数据在分组传输的平均往返时间**更长一些**。



福州大学
FUZHOU UNIVERSITY

59

自动重传请求 ARQ

计算机网络

- 通常 **A** 最终总是可以收到对所有发出的分组的确认。如果 **A** 不断重传分组但总是收不到确认，就说明通信线路太差，不能进行通信。
- 使用上述的确认和重传机制，我们就可以在不可靠的传输网络上实现可靠的通信。
- 像上述的这种可靠传输协议常称为**自动重传请求 ARQ (Automatic Repeat reQuest)**。意思是重传的请求是自动进行的，接收方不需要请求发送方重传某个出错的分组。



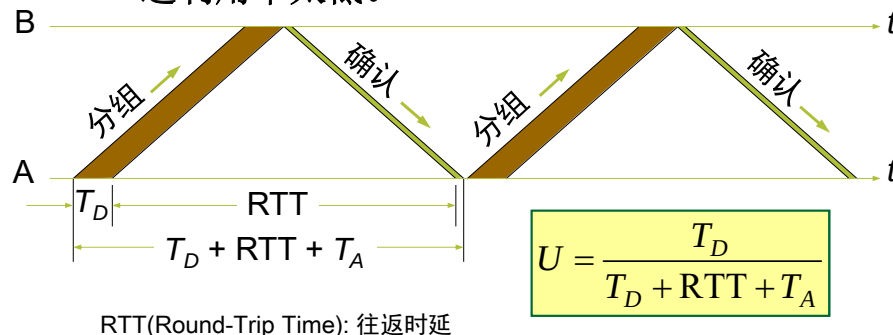
福州大学
FUZHOU UNIVERSITY

60

信道利用率

停止等待协议不适合发送时延远小于往返时延的情况！

- 停止等待协议的优点是简单，缺点是信道利用率太低。



福州大学
FUZHOU UNIVERSITY

61

连续ARQ: 流水线传输

连续不间断发送数据可能导致接收方或网络来不及处理

计算机网络

- 发送方可**连续发送**多个分组，不必每发完一个分组就停顿下来等待对方的确认。
- 由于信道上一一直有数据不间断地传送，这种传输方式可获得很高的信道利用率。

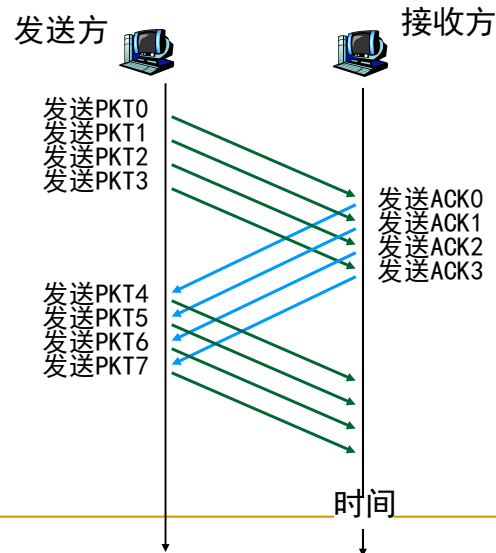


福州大学
FUZHOU UNIVERSITY

62

限制连续发送分组的数目

计算机网络



福州大学
FUZHOU UNIVERSITY

63

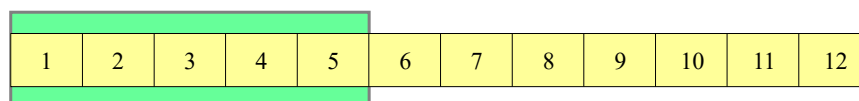
滑动窗口

计算机网络

通过设置发送窗口来限制

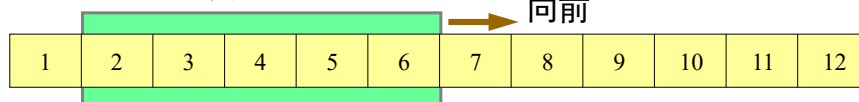
发送窗口大小是已发送
但还没有收到确认的最大
分组数

发送窗口



(a) 发送方维持发送窗口（发送窗口是 5）

发送窗口



(b) 收到一个确认后发送窗口向前滑动



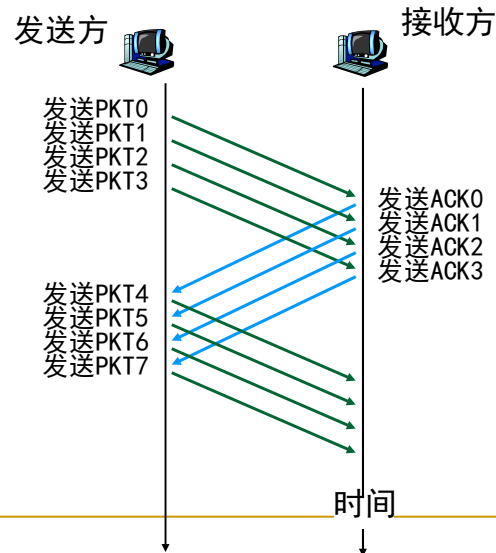
福州大学
FUZHOU UNIVERSITY

64

滑动窗口的作用

允许最多发送N个未收到ACK的分组，假设N为4

计算机网络

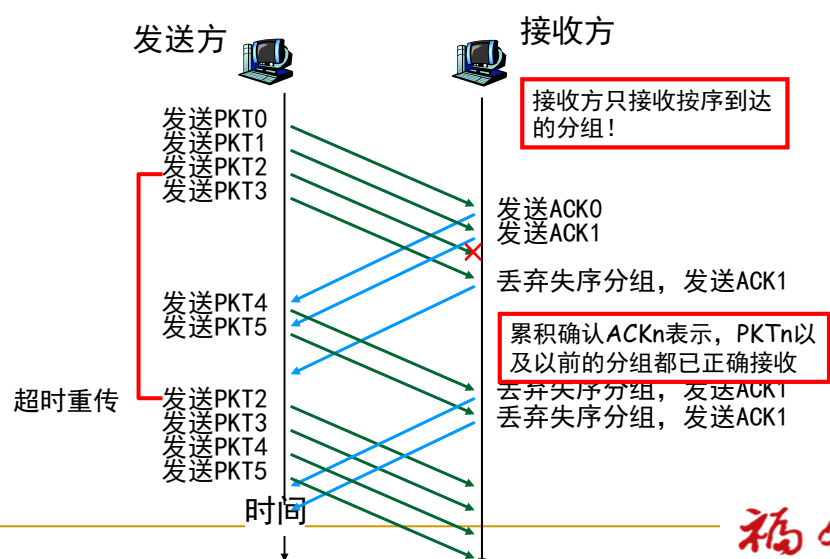


福州大学
FUZHOU UNIVERSITY

65

滑动窗口的作用

计算机网络



福州大学
FUZHOU UNIVERSITY

66

Go-back-N (回退 N)

计算机网络

- 如果发送方发送了前 5 个分组，而中间的第 3 个分组丢失了。这时接收方只能对前两个分组发出确认。发送方无法知道后面三个分组的下落，而只好把后面的三个分组都再重传一次。
- 这就叫做 **Go-back-N (回退 N)**，表示需要再退回来重传已发送过的 **N** 个分组。
- 可见当通信线路质量不好时，连续 **ARQ** 协议会带来负面的影响。



福州大学
FUZHOU UNIVERSITY

67

选择重传

计算机网络

- **GBN**协议存在一个缺点：一个分组的差错可能引起大量分组的重传，这些分组可能已经被接收方正确接收了，但由于未按序到达而被丢弃。
- 可设法只重传出现差错的分组。但必须加大接收窗口，以便先收下失序到达但仍然处在接收窗口中的哪些分组，等到所缺分组收齐后再一并送交上层。这就是**选择重传SR(Selective Repeat)**协议。

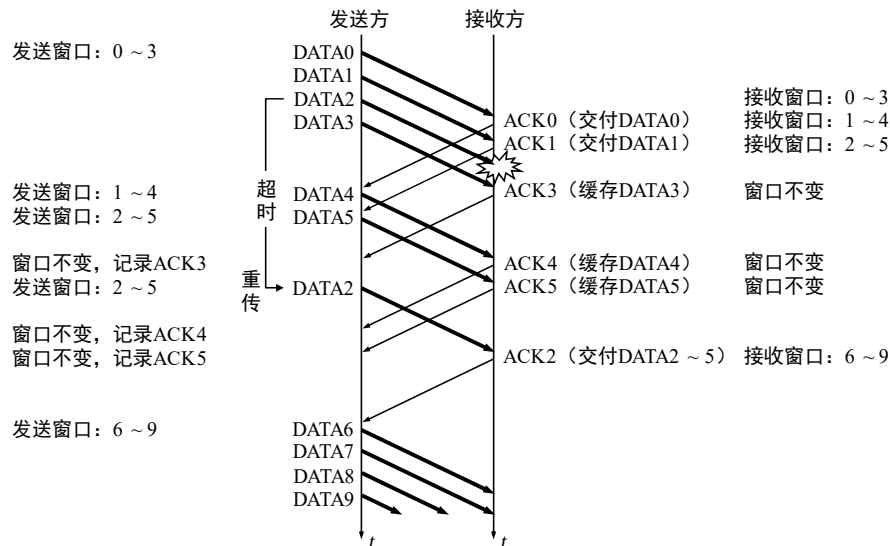


福州大学
FUZHOU UNIVERSITY

68

选择重传

计算机网络



福州大学
FUZHOU UNIVERSITY

69

TCP 可靠通信的具体实现

计算机网络

- TCP 连接的每一端都必须设有两个窗口——一个发送窗口和一个接收窗口。
- TCP 的可靠传输机制用字节的序号进行控制。TCP 所有的确认都是基于序号而不是基于报文段。
- TCP 两端的四个窗口经常处于动态变化之中。
- TCP连接的往返时间 RTT 也不是固定不变的。需要使用特定的算法估算较为合理的重传时间。



福州大学
FUZHOU UNIVERSITY

70

5.3.2 TCP 报文段结构

计算机网络

- **TCP** 虽然是面向字节流的，但 **TCP** 传送的数据单元却是报文段。
- 一个 **TCP** 报文段分为首部和数据两部分，而 **TCP** 的全部功能都体现在它首部中各字段的作用。
- **TCP** 报文段首部的 **前 20 个字节** 是固定的，后面有 **4n** 字节是根据需要而增加的选项 (n 是整数)。因此 **TCP 首部的最小长度是 20 字节**。

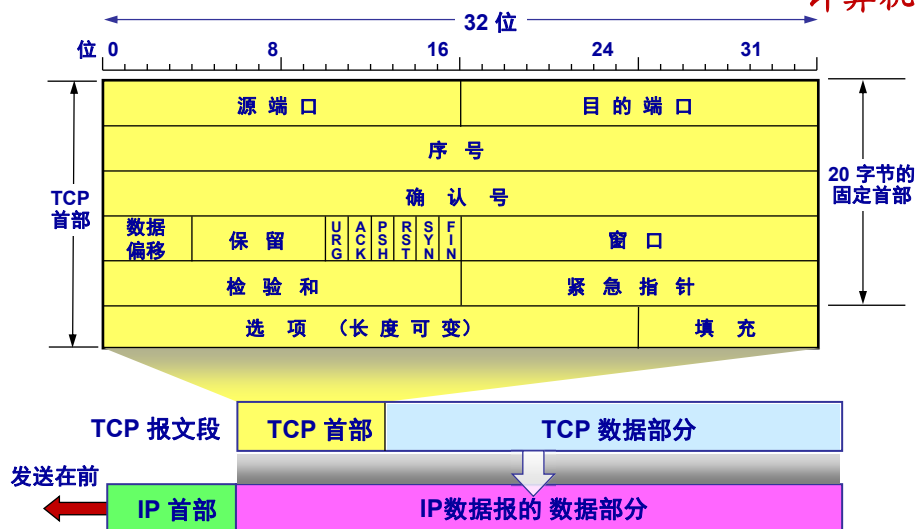


福州大学
FUZHOU UNIVERSITY

71

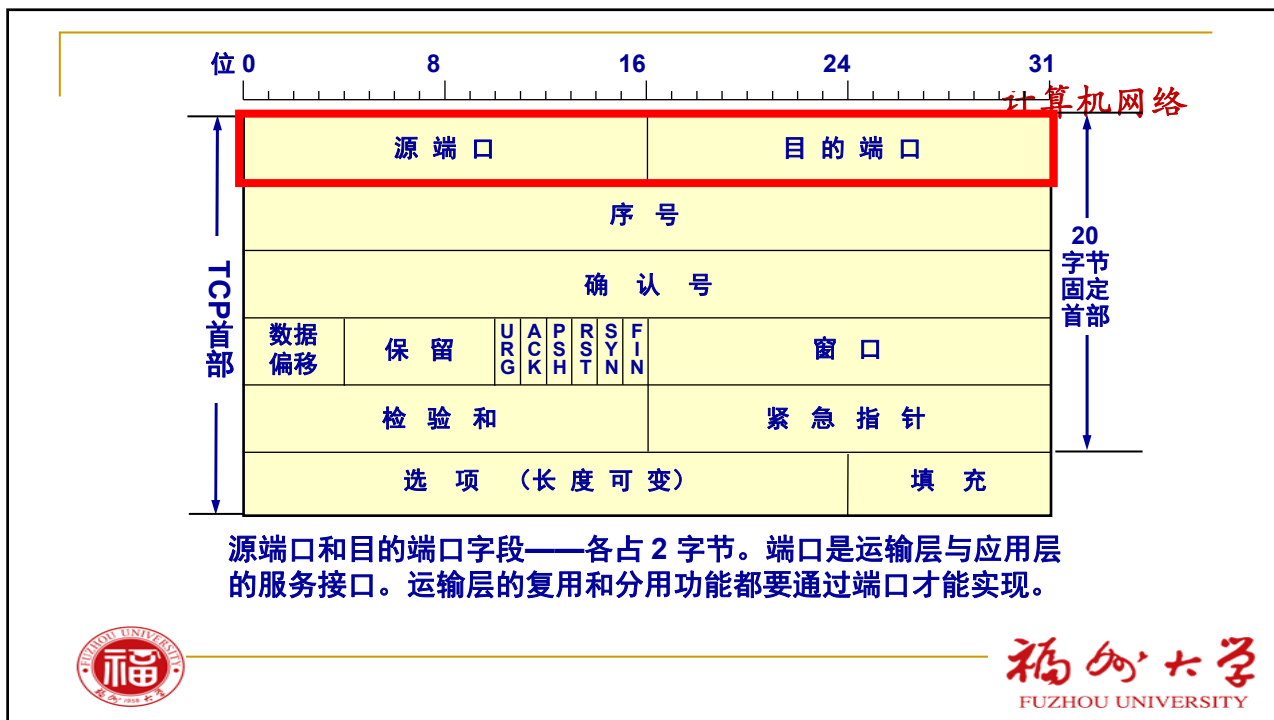
TCP 报文段的首部格式

计算机网络

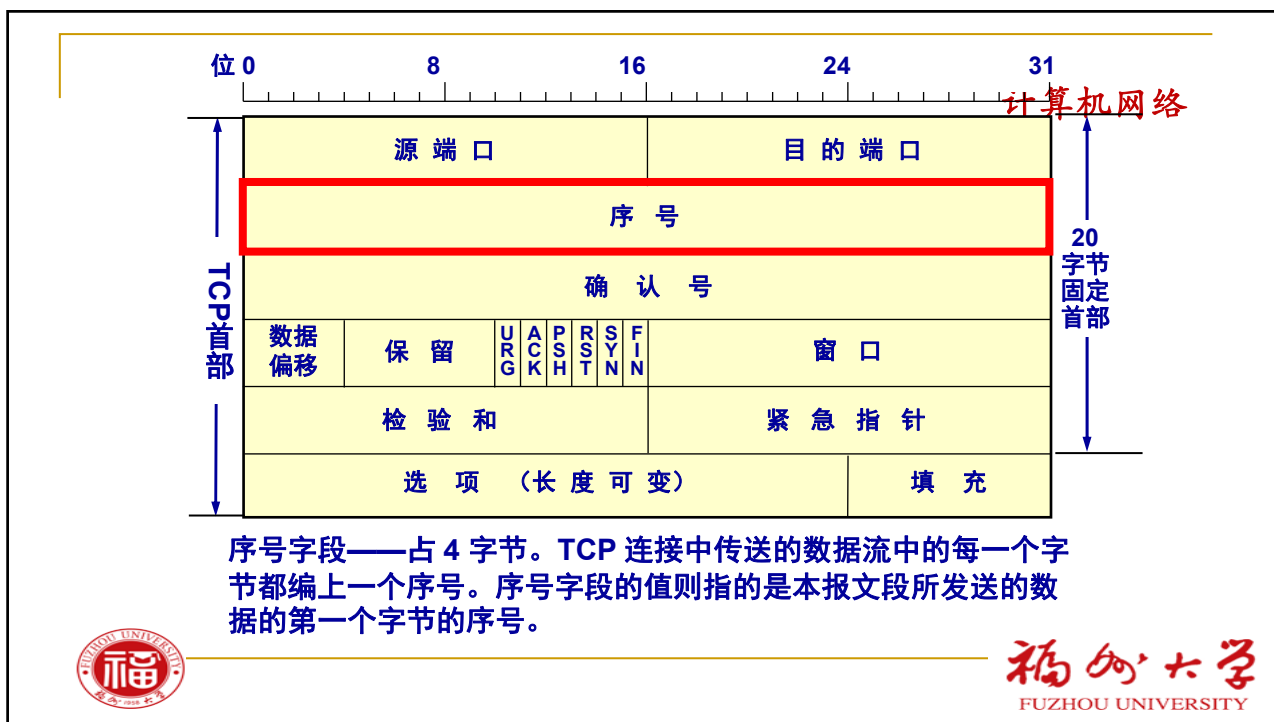


福州大学
FUZHOU UNIVERSITY

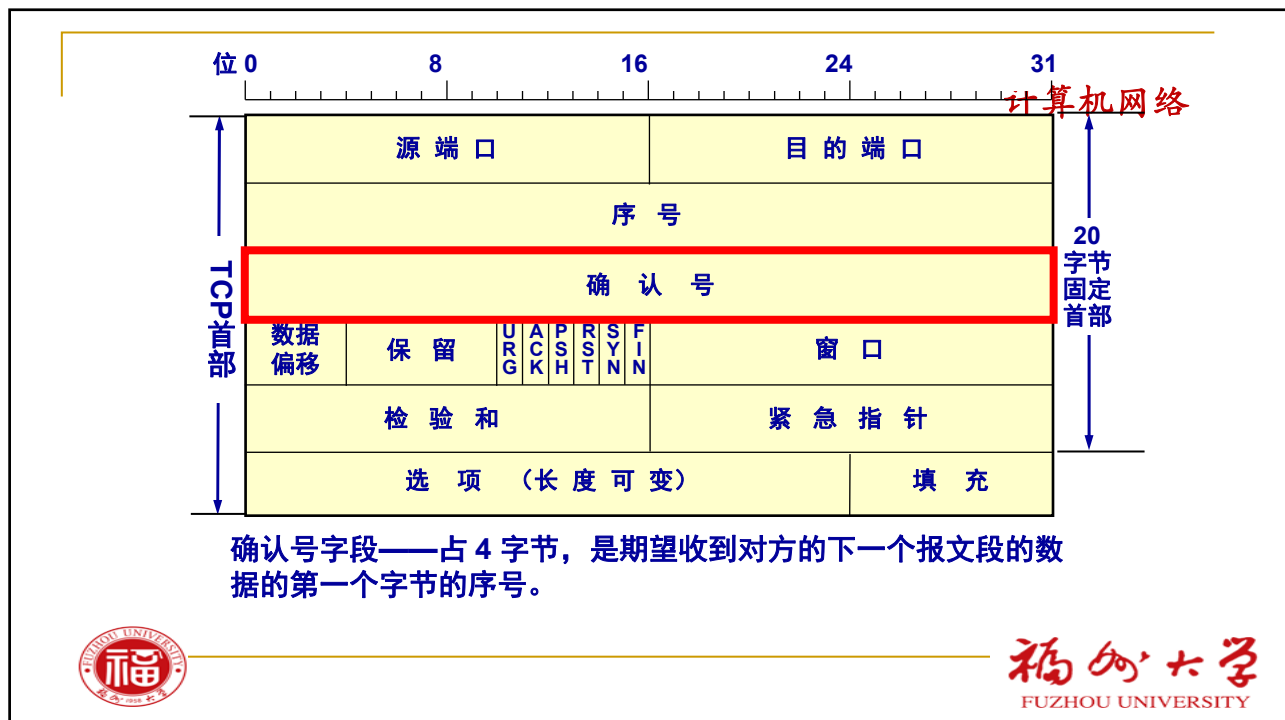
72



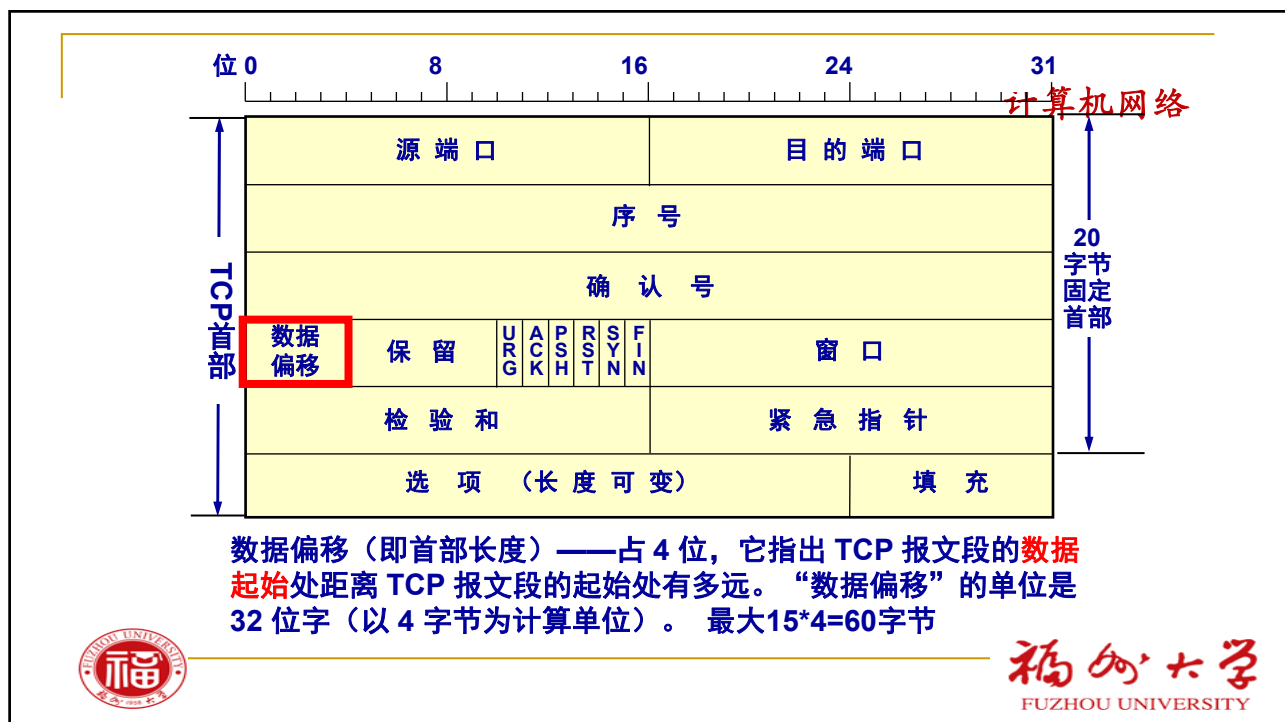
73



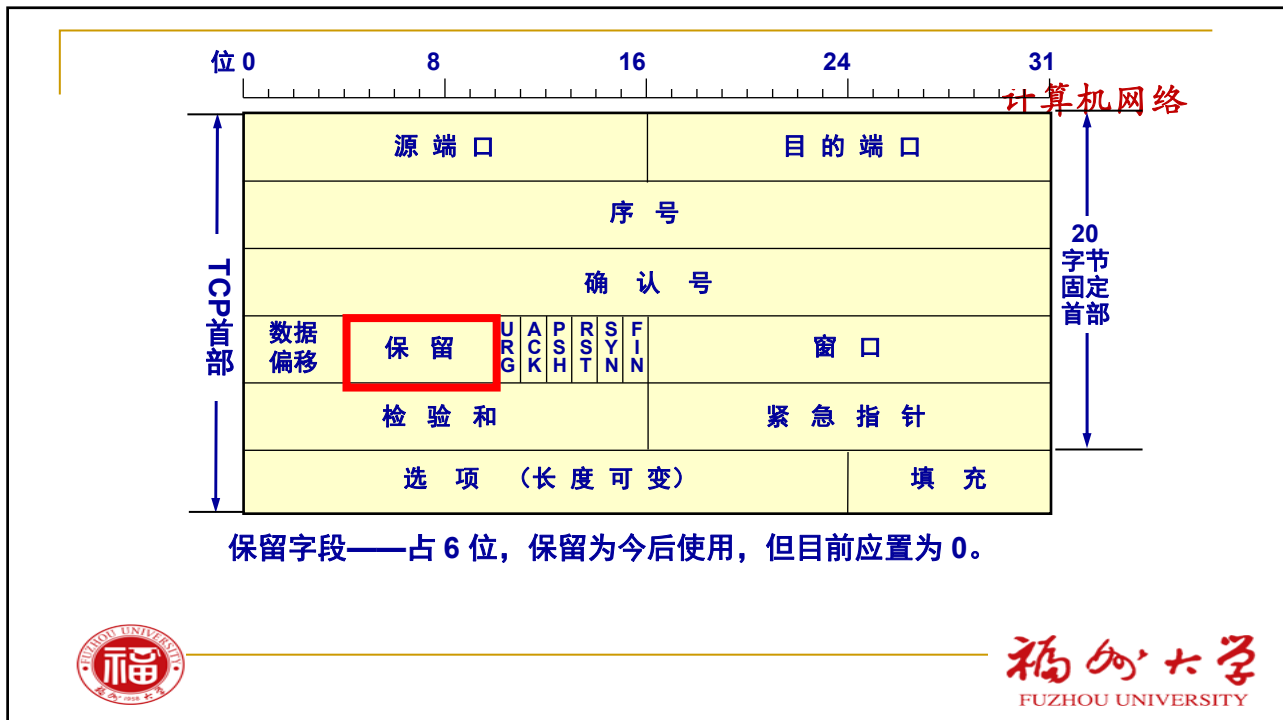
74



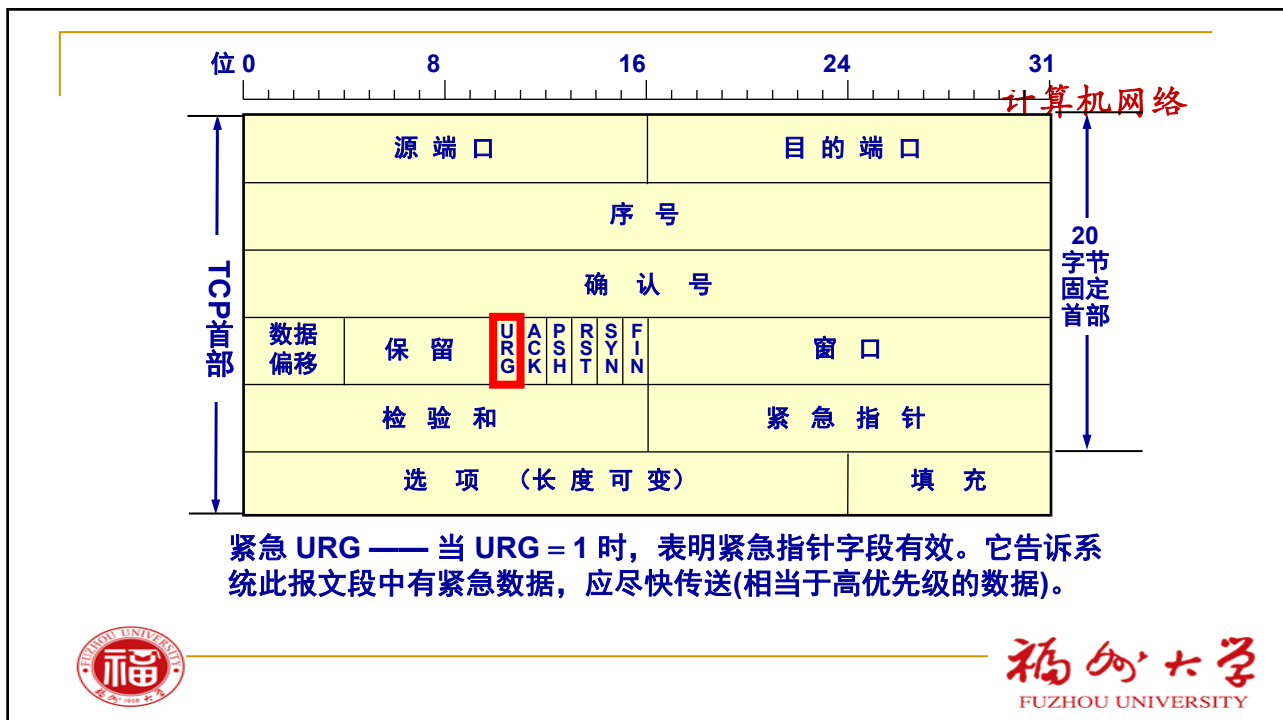
75



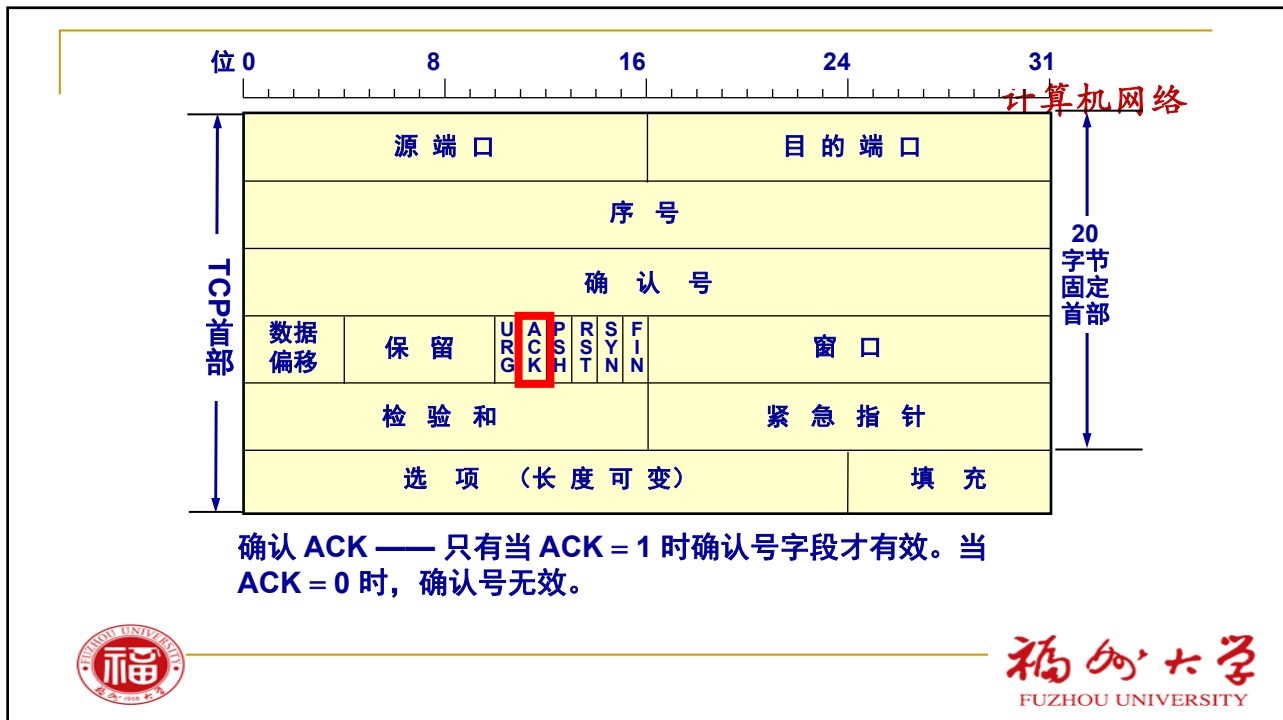
76



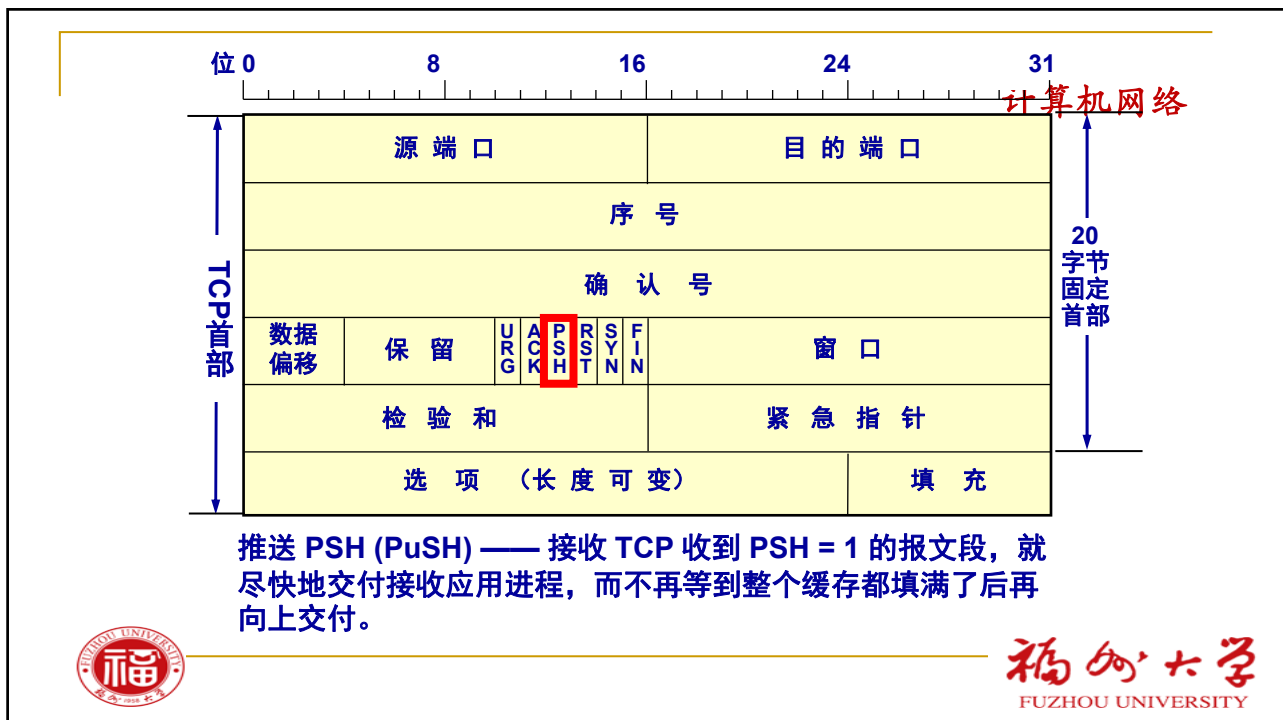
77



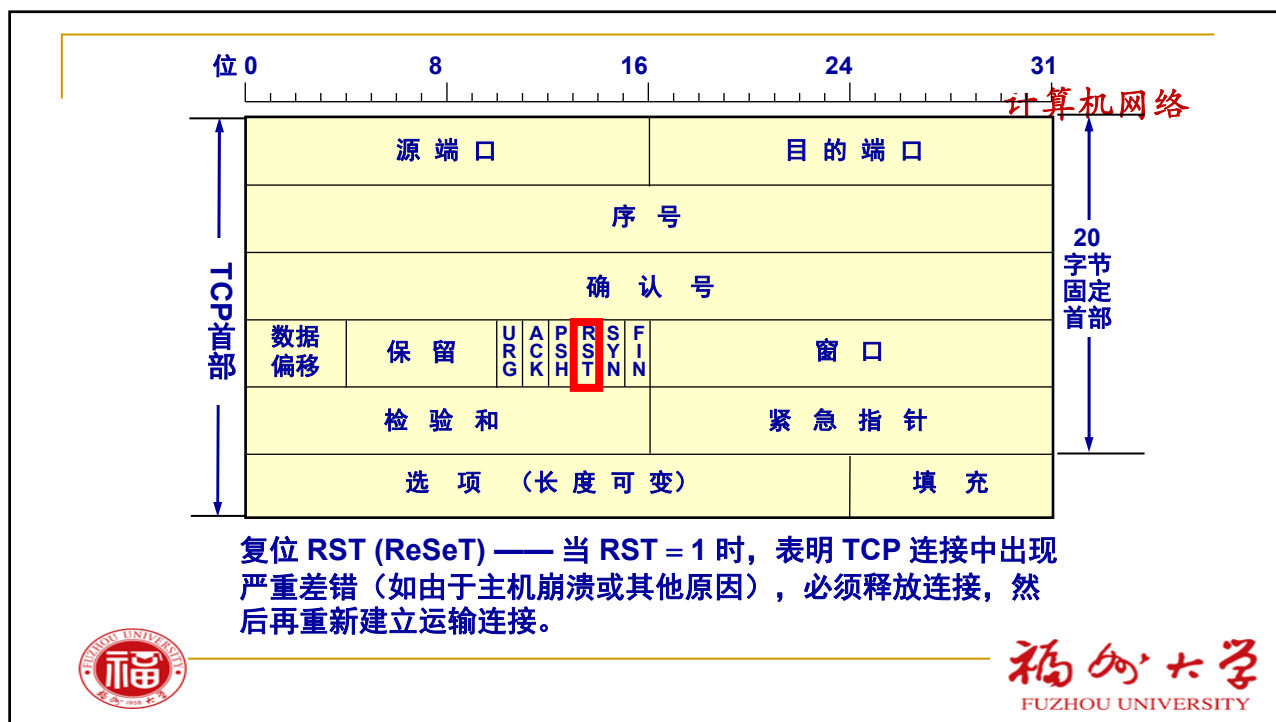
78



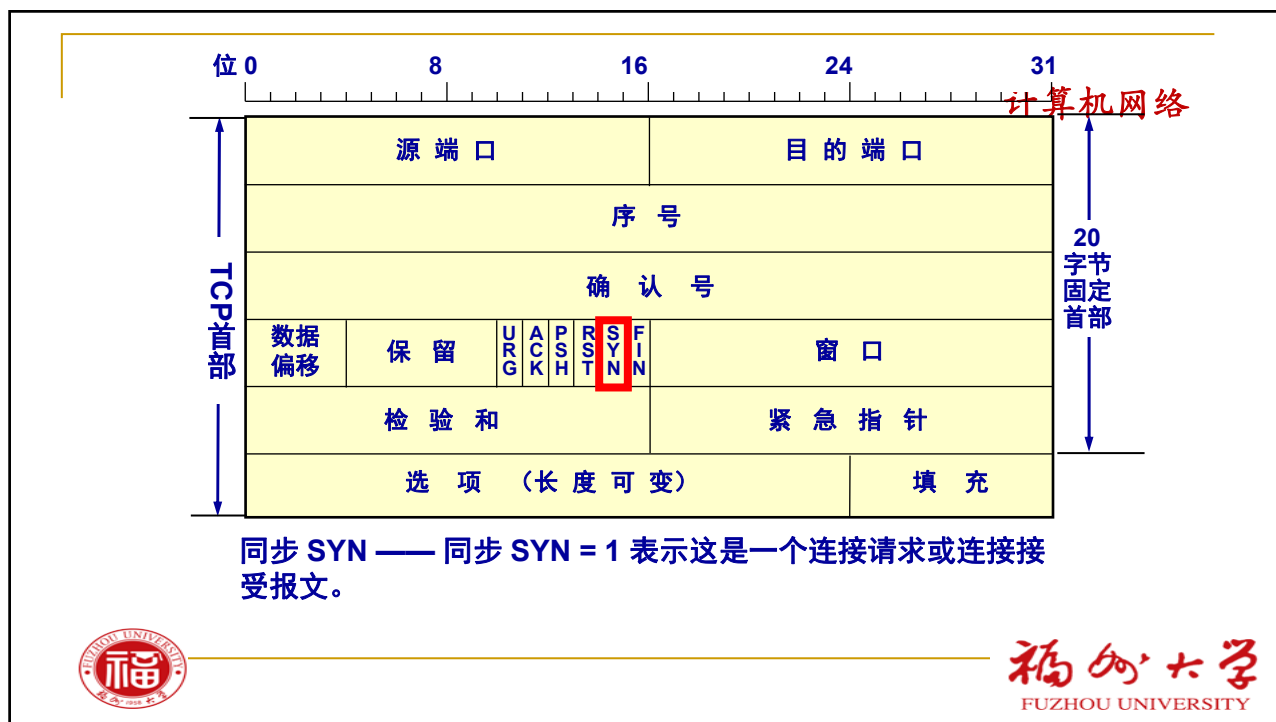
79



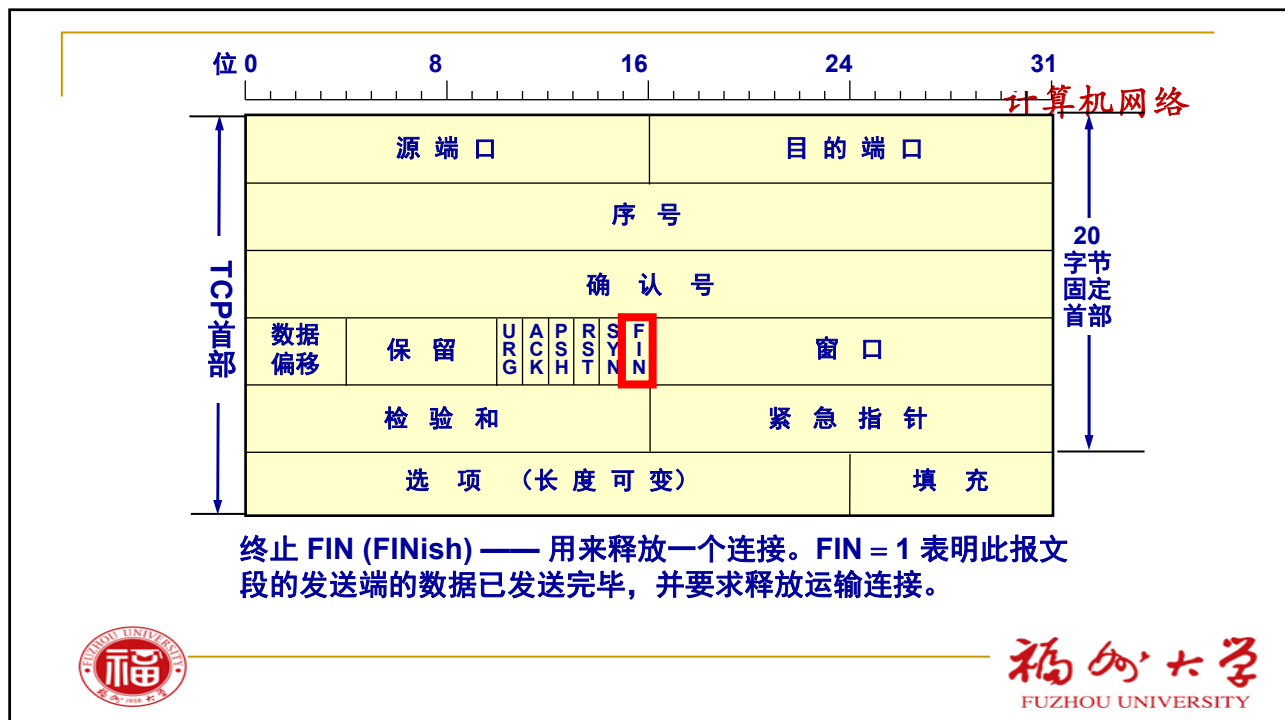
80



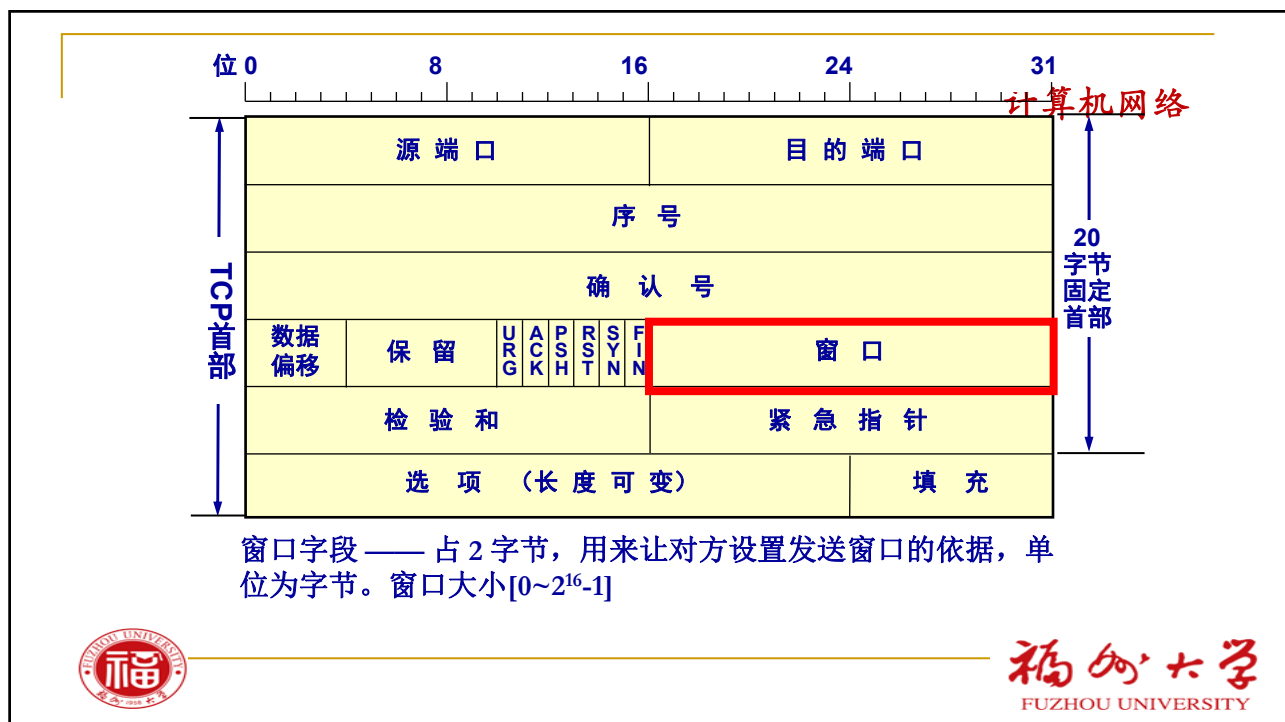
81



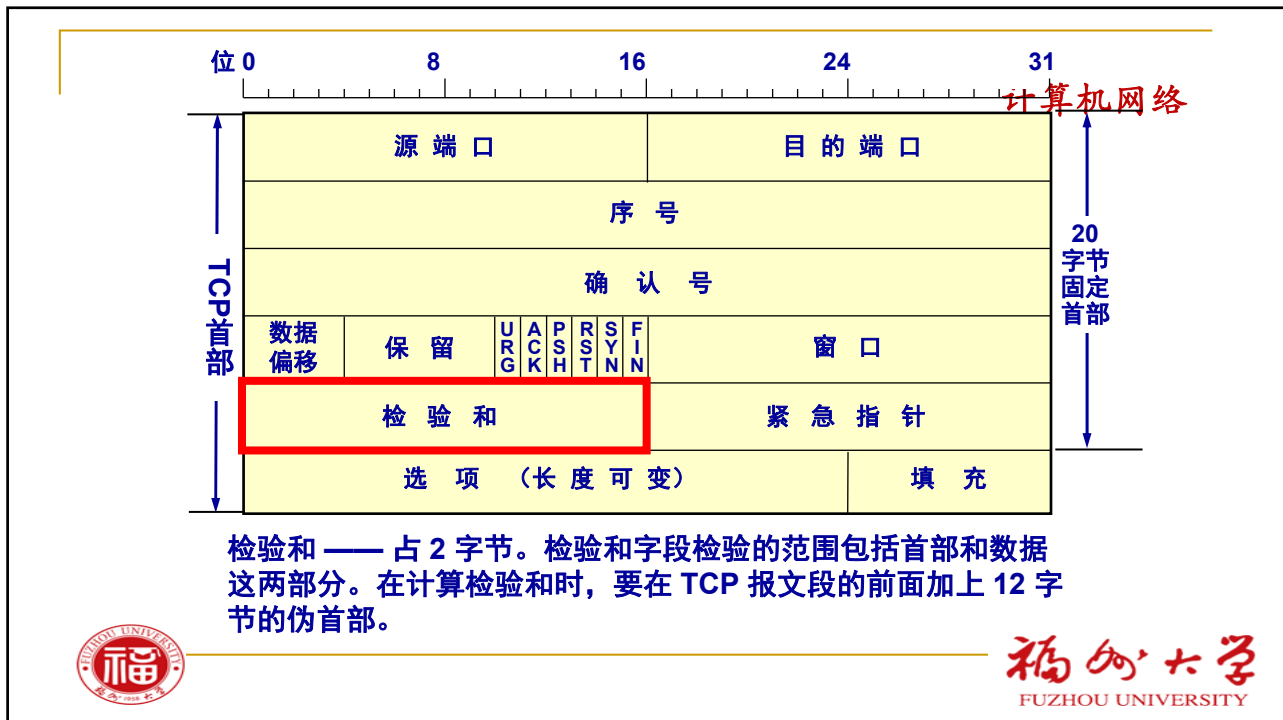
82



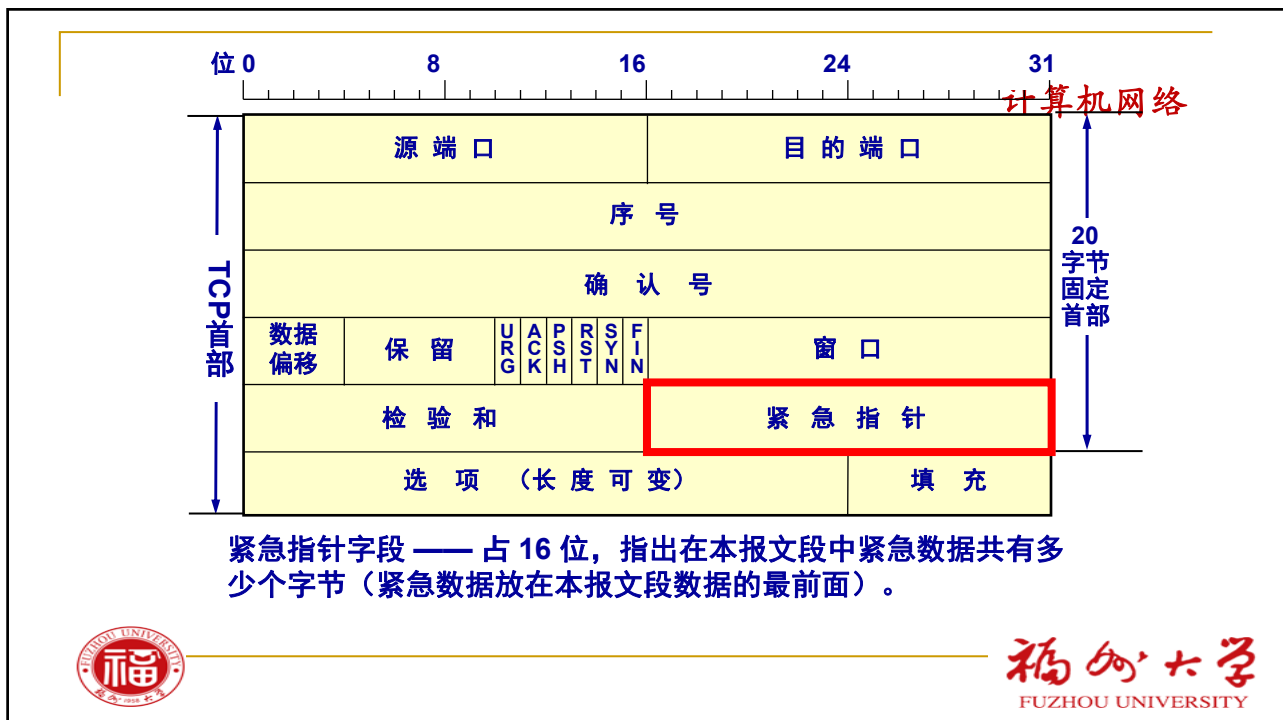
83



84



85



86

计算机网络

位 0 8 16 24 31

MSS (Maximum Segment Size)
 是 TCP 报文段中的**数据字段**的最大长度。
 数据字段加上 TCP 首部才等于整个的 TCP 报文段。
 所以，MSS是“TCP 报文段长度减去 TCP 首部长度的”。

选项字段 —— 长度可变。TCP 最初只规定了一种选项，即**最大报文段长度 MSS**。MSS 告诉对方 TCP：“我的缓存所能接收的报文段的数据字段的最大长度是 MSS 个字节。”




87

为什么要规定 MSS ?

计算机网络

- **MSS** 与接收窗口值没有关系。
- 若选择较小的 **MSS** 长度，网络的利用率就降低。
- 当 TCP 报文段只含有 **1 字节的数据**时，在 IP 层传输的数据报的开销至少有 **40 字节**(包括 TCP 报文段的首部和 IP 数据报的首部)。这样，对网络的利用率就不会超过 **1/41**。到了数据链路层还要加上一些开销。
- 若 **TCP 报文段非常长**，那么在 IP 层传输时就有可能要分解成多个短数据报片。在终点要把收到的各个短数据报片装配成原来的 TCP 报文段。当传输出错时还要进行重传。这些也都会使开销增大。




88

为什么要规定 MSS ?

计算机网络

- 因此, **MSS** 应尽可能大些, 只要在 **IP** 层传输时不需要再分片就行。
- 由于 **IP** 数据报所经历的路径是动态变化的, 因此在这条路径上确定的不需要分片的 **MSS**, 如果改走另一条路径就可能需要进行分片。
- 因此最佳的 **MSS** 是很难确定的。



福州大学
FUZHOU UNIVERSITY

89

其他选项

计算机网络

- **窗口扩大选项** ——占 3 字节, 其中有一个字节表示移位值 **S**。新的窗口值等于 **TCP** 首部中的窗口位数增大到 **(16 + S)**, 相当于把窗口值向左移动 **S** 位后获得实际的窗口大小。
- **时间戳选项** ——占 10 字节, 其中最主要的字段时间戳值字段 (4 字节) 和时间戳回送回答字段 (4 字节)。
- **选择确认选项**。



福州大学
FUZHOU UNIVERSITY

90



91

5.3.3 TCP 的可靠传输

计算机网络

- 因特网的网络层服务是不可靠的，即通过IP传送的数据可能出现差错、丢失、乱序或重复。
- **TCP**在IP的不可靠的尽最大努力服务的基础上实现了一种可靠数据传输服务，保证数据无差错、无丢失、按序和无重复的交付。
- 由于**TCP**下面的传输数据的互联网的结构非常复杂，因此不能采用最简单的停止等待协议来实现可靠传输。



福州大学
FUZHOU UNIVERSITY

92

1. 数据编号与确认

计算机网络

- **TCP** 协议是面向字节的。**TCP** 将所要传送的报文看成是字节组成的数据流，并使每一个字节对应于一个序号。
- 在连接建立时，双方要商定初始序号。**TCP** 每次发送的报文段的首部中的序号字段数值表示该报文段中的**数据部分的第一个字节的序号**。
- **TCP** 的确认是对接收到的数据的最高序号表示确认。接收方返回的确认号是已收到的数据的最高序号加 1。因此确认号表示**接收方期望下次收到的数据中的第一个数据字节的序号**。



福州大学
FUZHOU UNIVERSITY

93

1. 数据编号与确认（续）

计算机网络

- 由于**TCP**连接能提供全双工通信，因此通信中的每一方都不必专门发送确认报文段，而可以在传送数据时顺便把确认信息**捎带传送**。
- 为此，接收方在正确接收到数据时可能要等待一小段时间（不能超过**0.5秒**）再发送确认，若这段时间内有数据要发送给对方，则采用**捎带确认**。这样做可以提高传输效率。



福州大学
FUZHOU UNIVERSITY

94

1. 数据编号与确认（续）

计算机网络

- 接收方若收到有差错的报文段就丢弃（不发送否认信息）。
- 若收到重复的报文段，也要丢弃，但要发回（或捎带发回）确认信息。
- 若收到失序的报文段，可选择将失序报文段丢弃，或者先将其暂存于接收缓存内，待所缺序号的报文段收齐后再一起上交应用层。注意，不论采用哪种方法，接收方都要对已按序接收到的数据进行确认。



福州大学
FUZHOU UNIVERSITY

95

2. 以字节为单位的滑动窗口

计算机网络

- **TCP** 采用大小可变的滑动窗口进行流量控制。窗口大小的单位是字节。
- 在 **TCP** 报文段首部的窗口字段写入的数值就是当前给对方设置的发送窗口数值的上限。
- 发送窗口在连接建立时由双方商定。但在通信的过程中，接收方可根据自己的资源情况，随时动态地调整对方的发送窗口上限值(可增大或减小)。



福州大学
FUZHOU UNIVERSITY

96

2. 以字节为单位的滑动窗口

计算机网络

- **TCP** 的滑动窗口是以字节为单位的。
- 现假定 **A** 收到了 **B** 发来的确认报文段，其中窗口是 **20** 字节，而确认号是 **31**（这表明 **B** 期望收到的下一个序号是 **31**，而序号 **30** 为止的数据已经收到了）。
- 根据这两个数据，**A** 就构造出自己的发送窗口



福州大学
FUZHOU UNIVERSITY

97

- 根据 **B** 给出的窗口值，**A** 构造出自己的发送窗口。
- 发送窗口表示：在没有收到 **B** 的确认的情况下，**A** 可以连续把窗口内的数据都发送出去。
- 发送窗口里面的序号表示允许发送的序号。
- 显然，窗口越大，发送方就可以在收到对方确认之前连续发送更多的数据，因而可能获得更高的传输效率。

网络

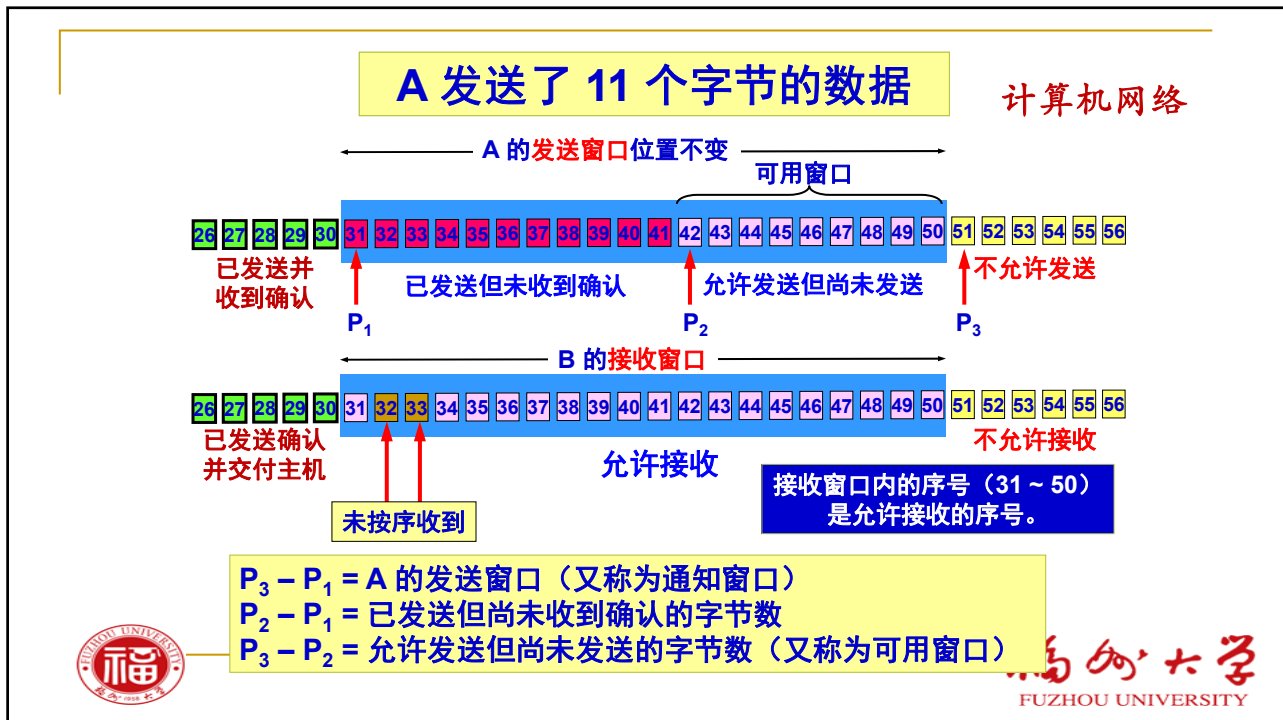


**TCP 标准强烈不赞成
发送窗口前沿向后收缩**

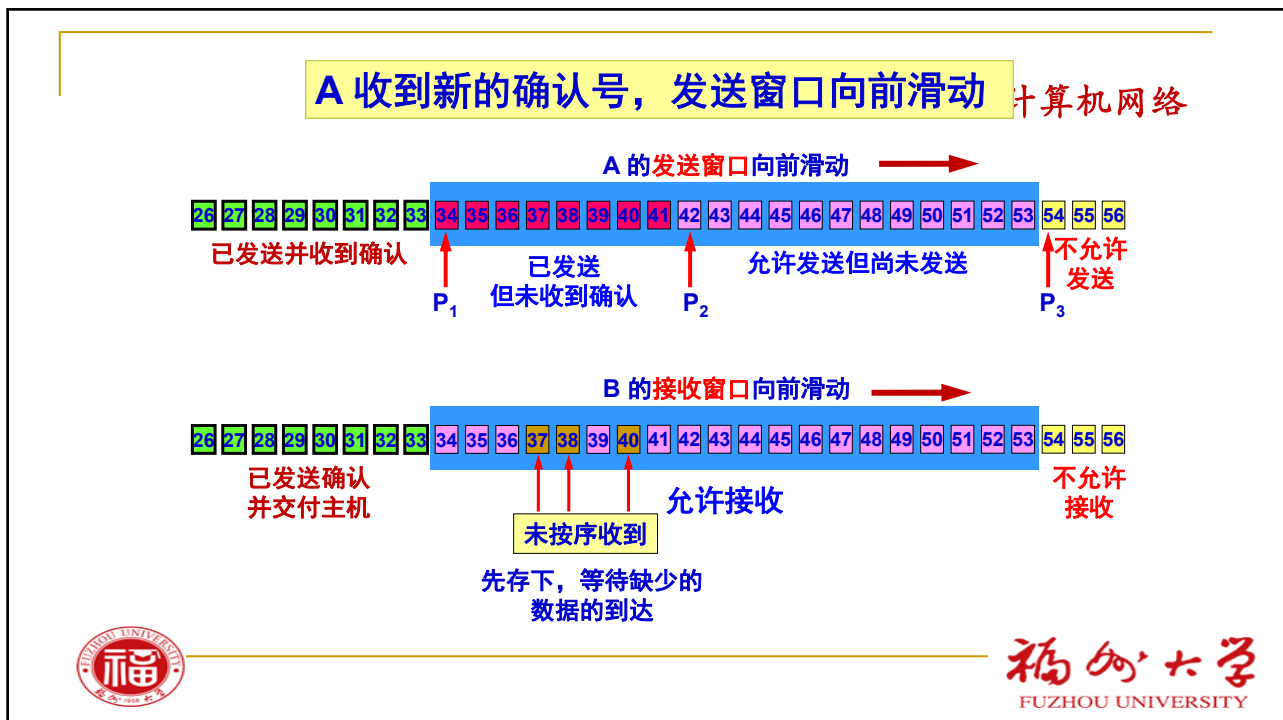


福州大学
FUZHOU UNIVERSITY

98

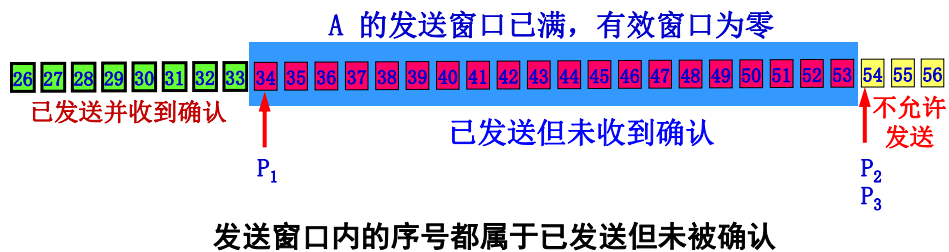


99



100

A 的发送窗口内的序号都已用完，但还没有再收到确认，必须停止发送。

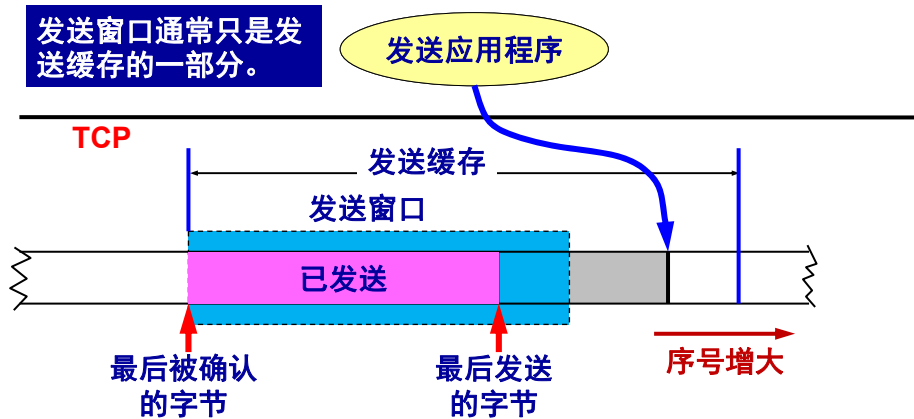


101

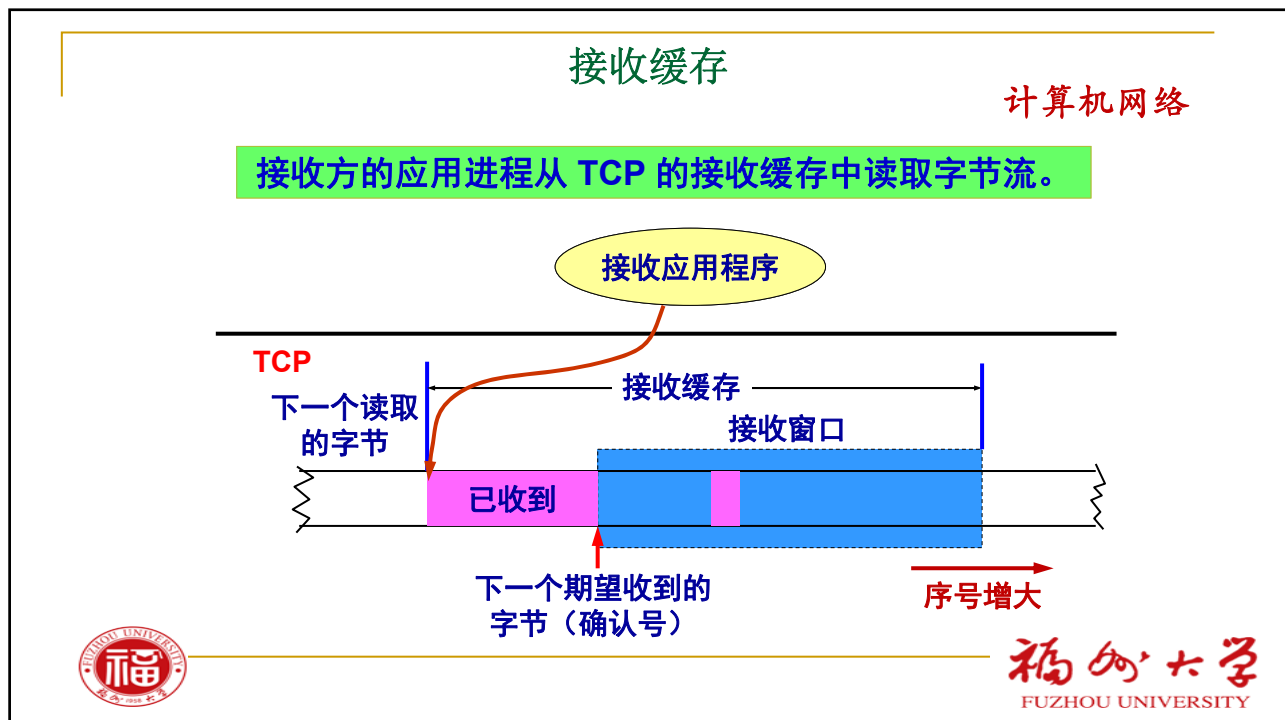
发送缓存

发送方的应用进程把字节流写入 TCP 的发送缓存。

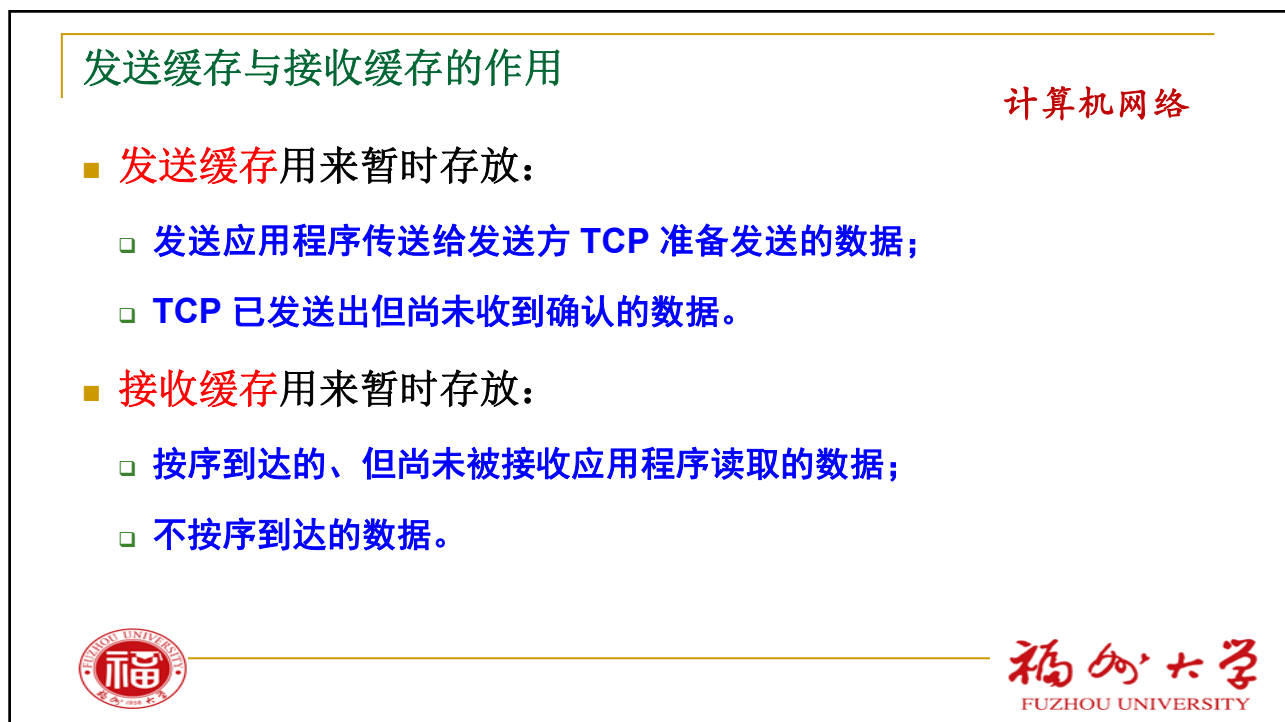
发送窗口通常只是发送缓存的一部分。



102



103



104

需要强调三点

计算机网络

- **第一**，A 的发送窗口并不总是和 B 的接收窗口一样大（因为有一定的时间滞后）。
- **第二**，TCP 标准没有规定对不按序到达的数据应如何处理。通常是先临时存放在接收窗口中，等到字节流中所缺少的字节收到后，再按序交付上层的应用进程。
- **第三**，TCP 要求接收方必须有累积确认的功能，这样可以减小传输开销。



福州大学
FUZHOU UNIVERSITY

105

接收方发送确认

计算机网络

- 接收方可以在合适的时候发送确认，也可以在自己有数据要发送时把确认信息顺便捎带上。
- 但请注意两点：
 - 第一，接收方不应过分推迟发送确认，否则会导致发送方不必要的重传，这反而浪费了网络的资源。
 - 第二，捎带确认实际上并不经常发生，因为大多数应用程序很少同时在两个方向上发送数据。



福州大学
FUZHOU UNIVERSITY

106

3. 超时重传时间的选择

计算机网络

- 重传机制是 **TCP** 中最重要和最复杂的问题之一。
- **TCP** 每发送一个报文段，就对这个报文段设置一次计时器。
- 只要计时器设置的重传时间到但还没有收到确认，就要重传这一报文段。
- 重传时间的选择是 **TCP** 最复杂的问题之一。



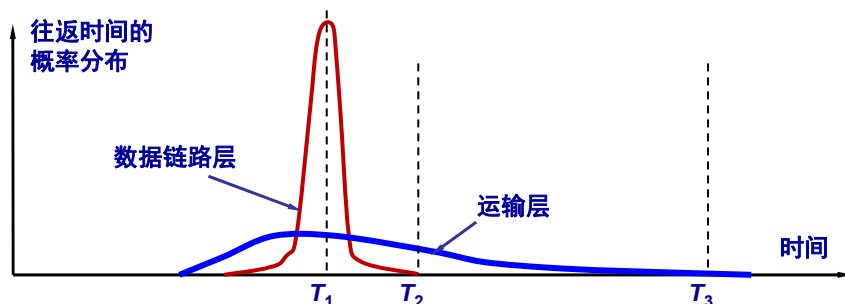
福州大学
FUZHOU UNIVERSITY

107

往返时延的方差很大

计算机网络

由于 **TCP** 的下层是一个互联网环境，**IP** 数据报所选择的路由变化很大。因而运输层的往返时间 (RTT) 的方差也很大。



福州大学
FUZHOU UNIVERSITY

108

TCP 超时重传时间设置

计算机网络

- 如果把超时重传时间设置得太短，就会引起很多报文段的不必要的重传，使网络负荷增大。
- 但若把超时重传时间设置得过长，则又使网络的空闲时间增大，降低了传输效率。
- **TCP 采用了一种自适应算法**，它记录一个报文段发出的时间，以及收到相应的确认的时间。这两个时间之差就是报文段的往返时间 **RTT**。



福州大学
FUZHOU UNIVERSITY

109

加权平均往返时间

计算机网络

- TCP 保留了 RTT 的一个**加权平均往返时间** RTT_s （这又称为**平滑的往返时间**）。
- 第一次测量到 RTT 样本时， RTT_s 值就取为所测量到的 RTT 样本值。以后每测量到一个新的 RTT 样本，就按下式重新计算一次 RTT_s ：

$$\text{新的}RTT_s = (1 - \alpha) \times (\text{旧的}RTT_s) + \alpha \times (\text{新的}RTT\text{样本})$$

- 式中， $0 \leq \alpha < 1$ 。若 α 很接近于零，表示 RTT 值更新较慢。若选择 α 接近于 1，则表示 RTT 值更新较快。
- RFC 2988 推荐的 α 值为 1/8，即 0.125。



福州大学
FUZHOU UNIVERSITY

110

超时重传时间 RTO

计算机网络

- **RTO (Retransmission Time-Out)** 应略大于上面得出的加权平均往返时间 RTT_s 。
- RFC 2988 建议使用下式计算 RTO:

$$RTO = RTT_s + 4 \times RTT_D$$

- RTT_D 是 **RTT** 的偏差的加权平均值。
- RFC 2988 建议这样计算 RTT_D 。第一次测量时, RTT_D 值取为测量到的 RTT 样本值的一半。在以后的测量中, 则使用下式计算加权平均的 RTT_D :

$$\begin{aligned} \text{新的 } RTT_D = & (1 - \beta) \times (\text{旧的 } RTT_D) \\ & + \beta \times |RTT_s - \text{新的 RTT 样本}| \end{aligned}$$



β 是个小于 1 的系数, 其推荐值是 1/4, 即 0.25。

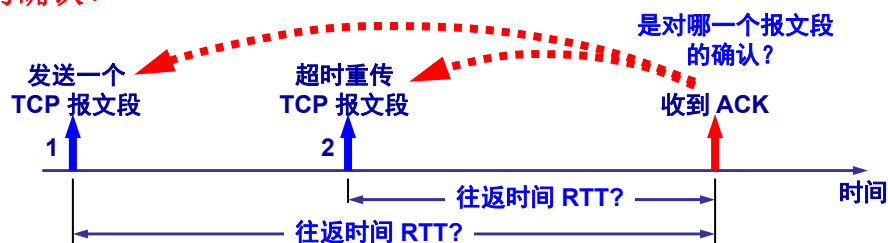
福州大学
FUZHOU UNIVERSITY

111

往返时间 (RTT) 的测量相当复杂

计算机网络

- TCP 报文段 1 没有收到确认。重传 (即报文段 2) 后, 收到了确认报文段 ACK。
- 如何判定此确认报文段是对原来的报文段 1 的确认, 还是对重传的报文段 2 的确认?



福州大学
FUZHOU UNIVERSITY

112

Karn 算法

计算机网络

- 在计算平均往返时间 **RTT** 时，只要报文段重传了，就不采用其往返时间样本。
- 这样得出的加权平均平均往返时间 **RTT_s** 和超时重传时间 **RTO** 就较准确。
- 但是，这又引起**新的问题**。当报文段的时延突然增大了很多时，在原来得出的重传时间内，不会收到确认报文段。于是就重传报文段。但根据 **Karn** 算法，不考虑重传的报文段的往返时间样本。这样，**超时重传时间就无法更新**。



福州大学
FUZHOU UNIVERSITY

113

修正的 Karn 算法

计算机网络

- 报文段每重传一次，就把 **RTO** 增大一些：

$$\text{新的 RTO} = \gamma \times (\text{旧的 RTO})$$

- 系数 γ 的典型值是 **2**。
- 当不再发生报文段的重传时，才根据报文段的往返时延更新平均往返时延 **RTT** 和超时重传时间 **RTO** 的数值。
- 实践证明，这种策略较为合理。



福州大学
FUZHOU UNIVERSITY

114

4. 快速重传

计算机网络

- 快速重传要求接收方每收到一个失序的报文段后就立即发出重复确认。
- 发送方只要一连收到三个重复确认就应当立即重传对方尚未收到的报文段。
- 不难看出，快重传并非取消重传计时器，而是在某些情况下可更早地重传丢失的报文段。

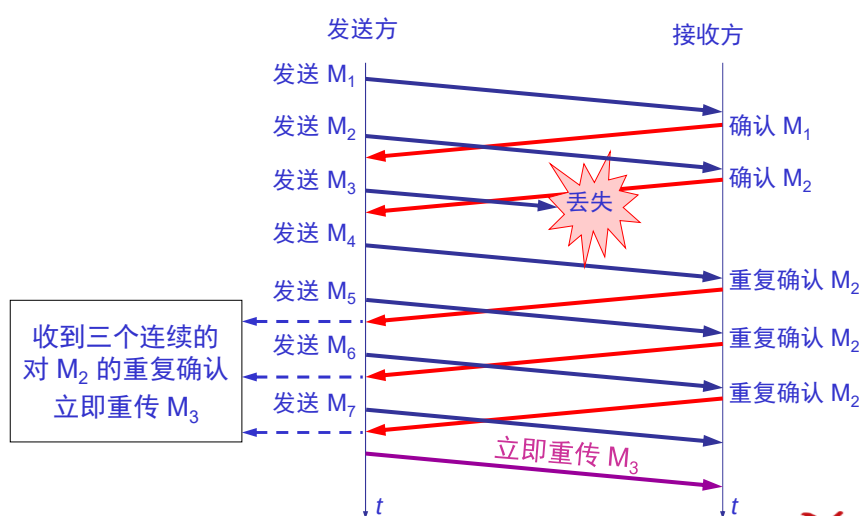


福州大学
FUZHOU UNIVERSITY

115

快速重传举例

计算机网络



福州大学
FUZHOU UNIVERSITY

116

5. 选择确认

计算机网络

- **问题：**若收到的报文段无差错，只是未按序号，中间还缺少一些序号的数据，那么能否设法只传送缺少的数据而不重传已经正确到达接收方的数据？
- 答案是可以的。**选择确认 SACK**
(**Selective ACK**) 就是一种可行的处理方法。



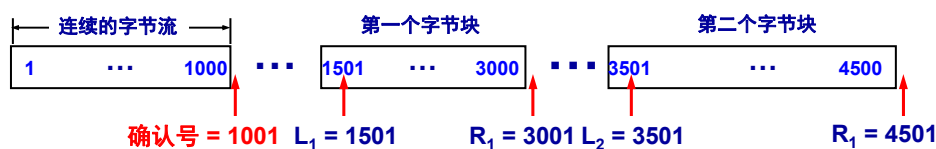
福州大学
FUZHOU UNIVERSITY

117

接收到的字节流序号不连续

计算机网络

TCP 的接收方在接收对方发送过来的数据字节流的序号不连续，结果就形成了一些不连续的字节块。



和前后字节不连续的每一个字节块都有**两个边界：左边界和右边界。**

- 第一个字节块的左边界 $L_1 = 1501$ ，但右边界 $R_1 = 3001$ 。左边界指出字节块的第一个字节的序号，但右边界减 1 才是字节块中的最后一个序号。
- 第二个字节块的左边界 $L_2 = 3501$ ，而右边界 $R_2 = 4501$ 。



福州大学
FUZHOU UNIVERSITY

118

5. 选择确认

计算机网络

- 接收方收到了和前面的字节流不连续的两个字节块。
- 如果这些字节的序号都在接收窗口之内，那么接收方就先收下这些数据，但要把这些信息准确地告诉发送方，使发送方不要再重复发送这些已收到的数据。



福州大学
FUZHOU UNIVERSITY

119

RFC 2018 的规定

计算机网络

- 如果要使用选择确认，那么在建立 TCP 连接时，就要在 TCP 首部的选项中加上“允许 SACK”的选项，而双方必须都事先商定好。
- 如果使用选择确认，那么原来首部中的“确认号字段”的用法仍然不变。只是以后在 TCP 报文段的首部中都增加了 SACK 选项，以便报告收到的不连续的字节块的边界。
- 由于首部选项的长度最多只有 40 字节，而指明一个边界就要用掉 4 字节
- 首部选项中需要增加“允许 SACK 选项字段”和选项所占字节，分别需要一个字节
- 因此在选项中最多只能指明 4 个字节块的边界信息。
- 如果 5 个字节块的边界信息：共 42 字节。
- 之后才能在数据传输阶段使用 SACK 选项字段进行选择确认。



福州大学
FUZHOU UNIVERSITY

120

5.3.4 流量控制

计算机网络

- 如果发送方把数据发送得过快，接收方就可能来不及接收，这就会造成数据的丢失。
- **流量控制(flow control)**就是让发送方的发送速率不要太快，既要让接收方来得及接收。
- 利用**滑动窗口**机制可以很方便地在 **TCP** 连接上实现流量控制。



福州大学
FUZHOU UNIVERSITY

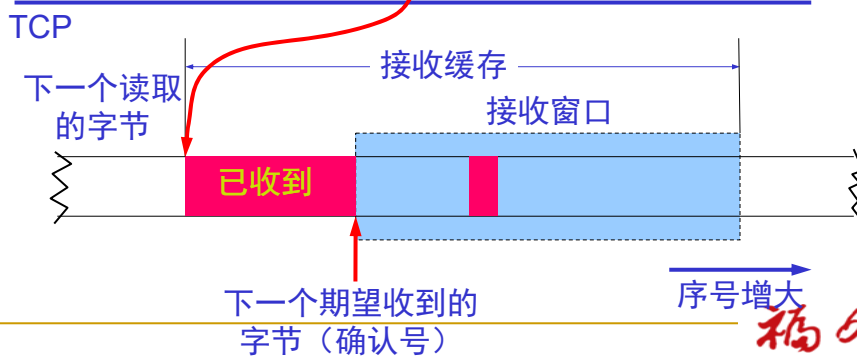
121

接收缓存

计算机网络

发送方的发送窗口
大小不能超过接收方的
接收窗口的大小！

接收应用程序



福州大学
FUZHOU UNIVERSITY

122

利用可变窗口进行流量控制举例

计算机网络

A 向 B 发送数据。在连接建立时，B 告诉 A：
“我的接收窗口 $rwnd = 400$ （字节）”。



福州大学
FUZHOU UNIVERSITY

123

可能发生死锁

计算机网络

- B 向 A 发送了零窗口的报文段后不久，B 的接收缓存又有了一些存储空间。于是 B 向 A 发送了 $rwnd = 400$ 的报文段。
- 但这个报文段在传送过程中**丢失**了。A 一直等待收到 B 发送的非零窗口的通知，而 B 也一直等待 A 发送的数据。
- 如果没有其他措施，这种**互相等待的死锁**局面将一直延续下去。
- 为了解决这个问题，TCP 为每一个连接设有一个**持续计时器 (persistence timer)**。



福州大学
FUZHOU UNIVERSITY

124

持续计时器

计算机网络

- **TCP** 为每一个连接设有一个**持续计时器** (persistence timer) 。
- 只要 **TCP** 连接的一方收到对方的**零窗口**通知，就启动该持续计时器。
- 若持续计时器设置的时间到期，就发送一个零窗口探测报文段（仅携带 1 字节的数据），而对方就在确认这个探测报文段时给出了现在的窗口值。
- 若窗口仍然是零，则收到这个报文段的一方就重新设置持续计时器。
- 若窗口不是零，则死锁的僵局就可以打破了。



福州大学
FUZHOU UNIVERSITY

125

必须考虑传输效率

计算机网络

- 可以用不同的机制来控制 **TCP** 报文段的发送时机：
 - **第一种机制**是 **TCP** 维持一个变量，它等于最大报文段长度 **MSS**。只要缓存中存放的数据达到 **MSS** 字节时，就组装成一个 **TCP** 报文段发送出去。
 - **第二种机制**是由发送方的应用进程指明要求发送报文段，即 **TCP** 支持的**推送** (push)操作。
 - **第三种机制**是发送方的一个计时器期限到了，这时就把当前已有的缓存数据装入报文段（但长度不能超过 **MSS**）发送出去。
- 如何控制 **TCP** 发送报文段的时机仍然是一个较为复杂的问题。



福州大学
FUZHOU UNIVERSITY

126

必须考虑传输效率

计算机网络

- 发送方 **TCP** 每次接收到一字节的数据后就发送。
- 这样，发送一个字节需要形成 **41** 字节长的 **IP** 数据报。若接收方确认，并回送这一字节，就需传送总长度为 **162** 字节共 **4** 个报文段。效率很低。
- **解决方法**：使用 **Nagle** 算法。



福州大学
FUZHOU UNIVERSITY

127

Nagle算法

计算机网络

- 若发送应用进程把要发送的数据逐个字节地送到 **TCP** 的发送缓存，则发送方就把第一个数据字节先发送出去，把后面到达的数据字节都缓存起来。
- 当发送方收到对第一个数据字符的确认后，再把发送缓存中的所有数据组装成一个报文段发送出去，同时继续对随后到达的数据进行缓存。
- 只有在收到对前一个报文段的确认后才继续发送下一个报文段。
- 当到达的数据已达到发送窗口大小的一半或已达到报文段的最大长度时，就立即发送一个报文段。



福州大学
FUZHOU UNIVERSITY

128

接收方糊涂窗口综合症

计算机网络

- 当接收方的 **TCP** 缓冲区已满，接收方会向发送方发送窗口大小为 **0** 的报文。
- 若此时接收方的应用进程以交互方式每次只读取一个字节，于是接收方又发送窗口大小为一个字节的更新报文，发送方应邀发送一个字节的的数据（发送的 **IP** 数据报是 **41** 字节长），于是接收窗口又满了，如此循环往复。
- **解决方法**：让接收方等待一段时间，使得或者接收缓存已有足够空间容纳一个最长的报文段，或者等到接收缓存已有一半空闲的空间。只要出现这两种情况之一，接收方就发出确认报文，并向发送方通知当前的窗口大小。



福州大学
FUZHOU UNIVERSITY

129

5.3.5 TCP 的连接管理

计算机网络

■ 1. TCP的连接建立

- 运输连接就有三个阶段，即：连接建立、数据传送和连接释放。运输连接的管理就是使运输连接的建立和释放都能正常地进行。
- 连接建立过程中要解决以下三个问题：
 - 要使每一方能够确知对方的存在。
 - 要允许双方协商一些参数（如最大报文段长度，最大窗口大小，服务质量等）。
 - 能够对运输实体资源（如缓存大小，连接表中的项目等）进行分配。



福州大学
FUZHOU UNIVERSITY

130

客户-服务器方式

计算机网络

- **TCP**连接的建立采用**客户服务器方式**。
- 主动发起连接建立的应用进程叫做**客户(client)**,
- 被动等待连接建立的应用进程叫做**服务器(server)**。



福州大学
FUZHOU UNIVERSITY

131

TCP 的连接建立

计算机网络

- **TCP** 建立连接的过程叫做**握手**。
- 握手需要在客户和服务器之间交换三个 **TCP** 报文段。称之为**三报文握手**。
- 采用**三报文握手**主要是为了防止已失效的连接请求报文段突然又传送到了, 因而产生错误。

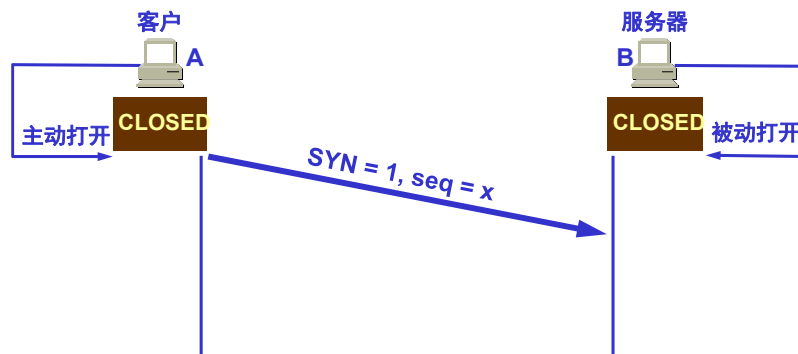


福州大学
FUZHOU UNIVERSITY

132

TCP 的连接建立：采用三报文握手

计算机网络



A 的 TCP 向 B 发出连接请求报文段，其首部中的同步位 $SYN = 1$ ，并选择序号 $seq = x$ ，表明传送数据时的第一个数据字节的序号是 x 。

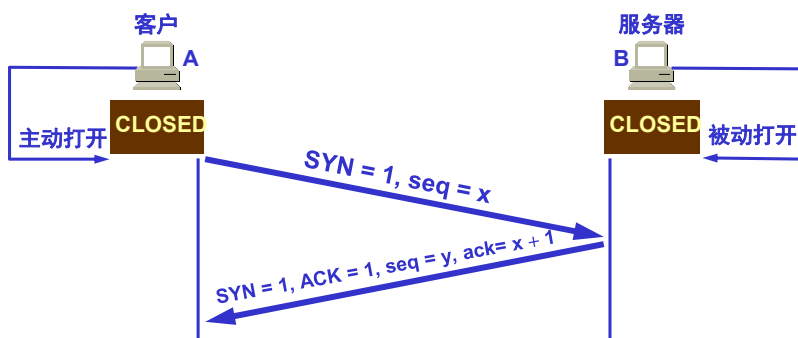


福州大学
FUZHOU UNIVERSITY

133

TCP 的连接建立：采用三报文握手

计算机网络



- B 的 TCP 收到连接请求报文段后，如同意，则发回确认。
- B 在确认报文段中应使 $SYN = 1$ ，使 $ACK = 1$ ，其确认号 $ack = x + 1$ ，自己选择的序号 $seq = y$ 。

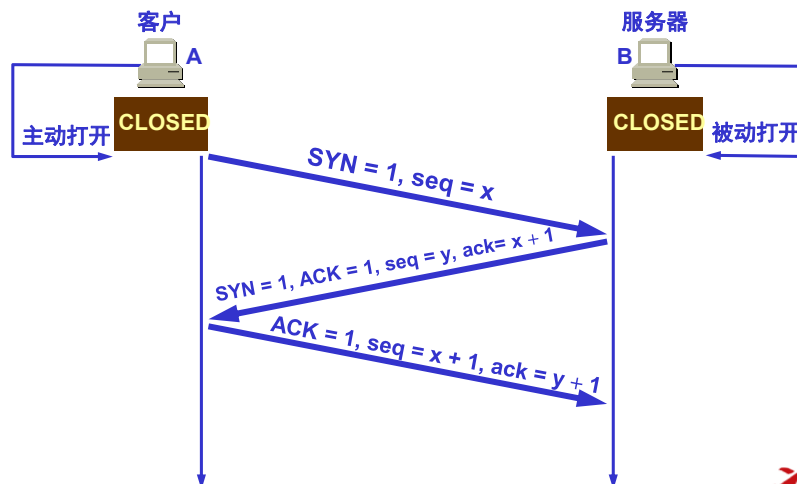


福州大学
FUZHOU UNIVERSITY

134

- A 收到此报文段后向 B 给出确认，其 ACK = 1，确认号 $ack = y + 1$ 。
- A 的 TCP 通知上层应用进程，连接已经建立。

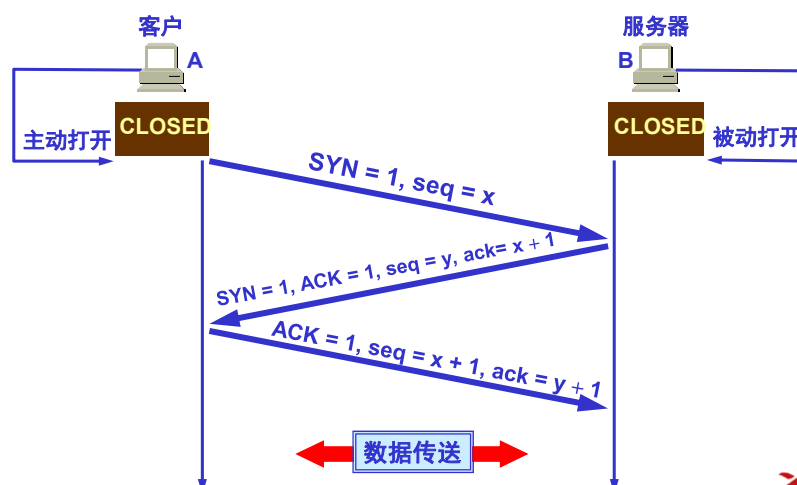
计算机网络

福州大学
FUZHOU UNIVERSITY

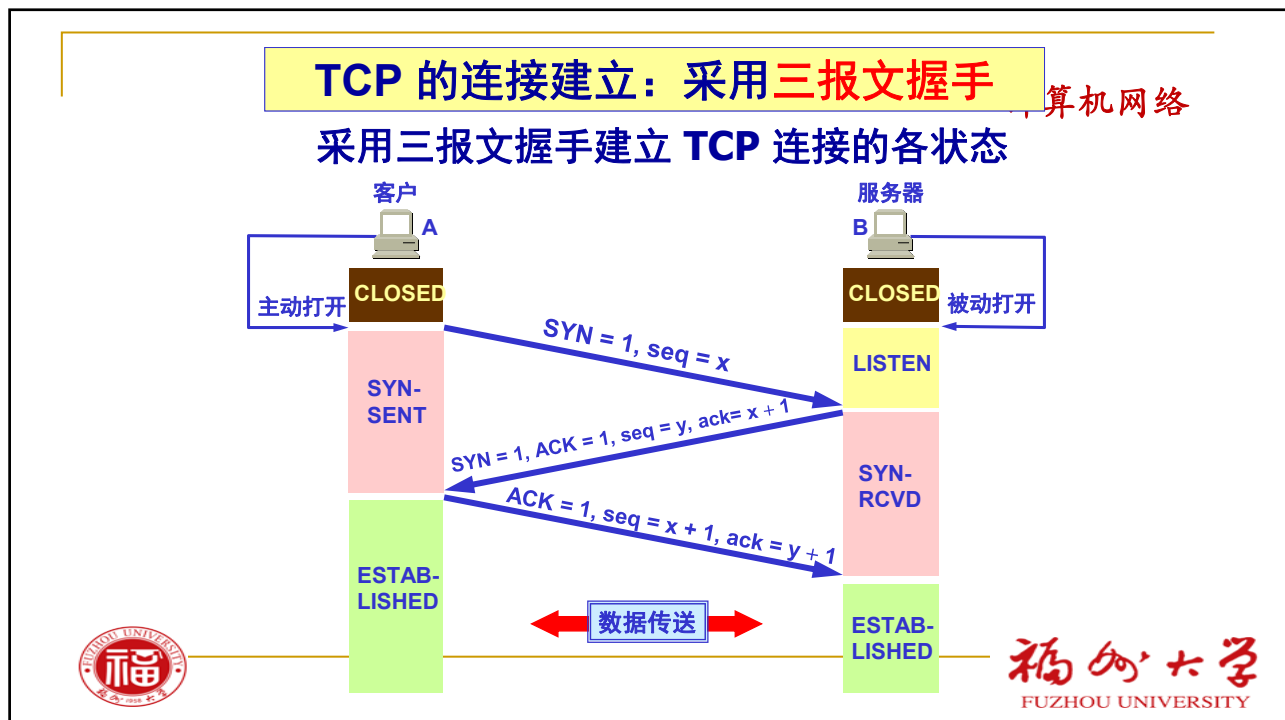
135

- B 的 TCP 收到主机 A 的确认后，也通知其上层应用进程：TCP 连接已经建立。

计算机网络

福州大学
FUZHOU UNIVERSITY

136



137

建立 TCP 连接 计算机网络

- **A 的 TCP 向 B 发出连接请求报文段**，其首部中的同步位 **SYN = 1**，并选择序号 **x**，表明下一个报文段的第一个数据字节的序号是 **x + 1**。
- **B 的 TCP 收到连接请求报文段后**，如同意，则发回确认，在确认报文段中使 **SYN = 1** 和 **ACK = 1**，其确认号应为 **ack = x + 1**，并选择序号 **seq = y**。
- **A 收到此报文段后**，向 **B** 给出确认，其 **ACK = 1**，序号应为 **seq = x + 1**，确认号应为 **ack = y + 1**。
- **A 的 TCP 通知上层应用进程**，连接已经建立。
- 当运行服务器进程的主机 **B 的 TCP 收到主机 A 的确认后**，也通知其上层应用进程，连接已经建立。

 **福州大学**
FUZHOU UNIVERSITY

138

2. TCP 的连接释放

计算机网络

- TCP 连接释放过程比较复杂。
- 数据传输结束后，通信的双方都可释放连接。
- TCP 连接释放过程是**四报文握手**。

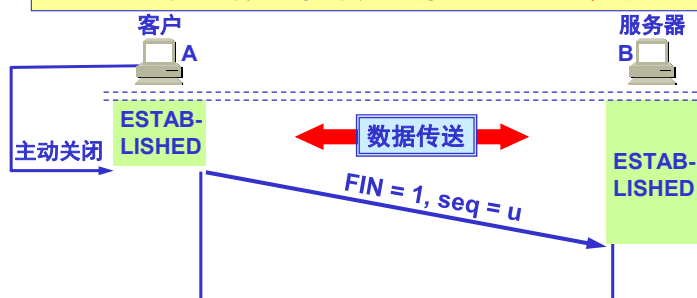


福州大学
FUZHOU UNIVERSITY

139

TCP 的连接释放：采用**四报文握手**

计算机网络



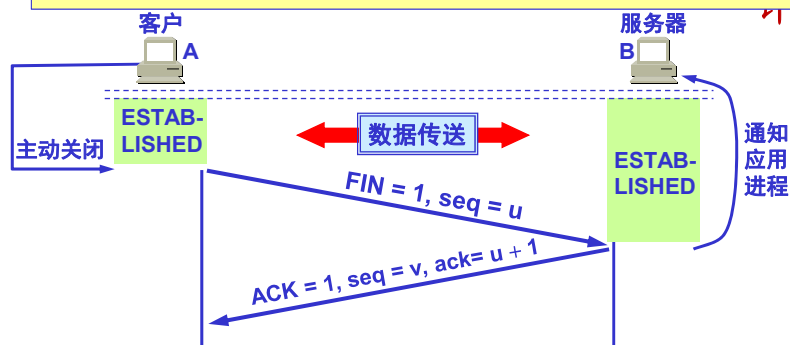
- 数据传输结束后，通信的双方都可释放连接。
- 现在 A 的应用进程先向其 TCP 发出连接释放报文段，并停止再发送数据，主动关闭 TCP 连接。
- A 把连接释放报文段首部的 $FIN = 1$ ，其序号 $seq = u$ ，等待 B 的确认。



福州大学
FUZHOU UNIVERSITY

140

TCP 的连接释放：采用四报文握手



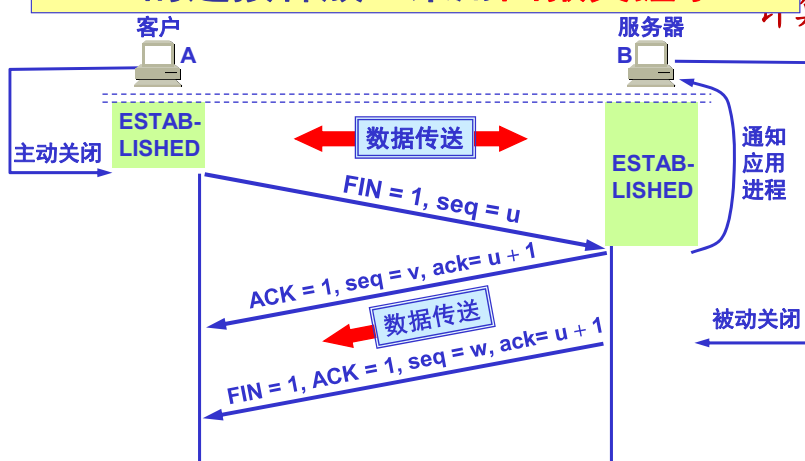
- B 发出确认，确认号 $ack = u + 1$ ，而这个报文段自己的序号 $seq = v$ 。
- TCP 服务器进程通知高层应用进程。
- 从 A 到 B 这个方向的连接就释放了，TCP 连接处于半关闭状态。B 若发送数据，A 仍要接收。



福州大学
FUZHOU UNIVERSITY

141

TCP 的连接释放：采用四报文握手



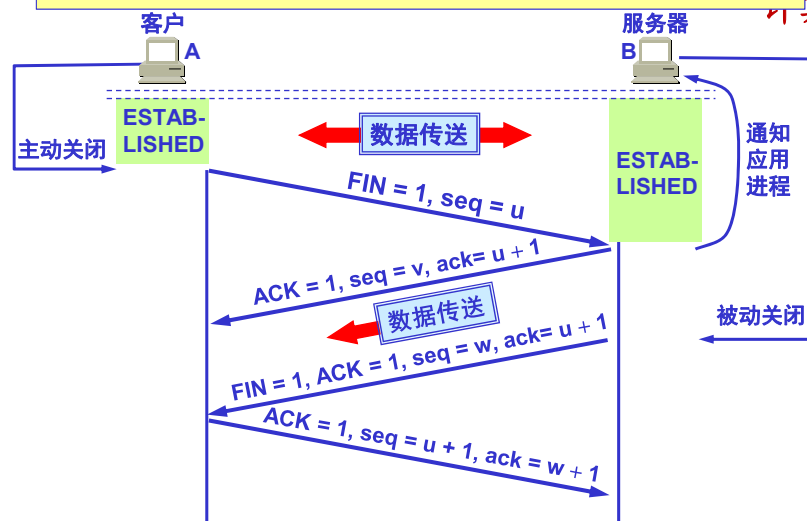
- 若 B 已经没有要向 A 发送的数据，其应用进程就通知 TCP 释放连接。



福州大学
FUZHOU UNIVERSITY

142

TCP 的连接释放：采用四报文握手



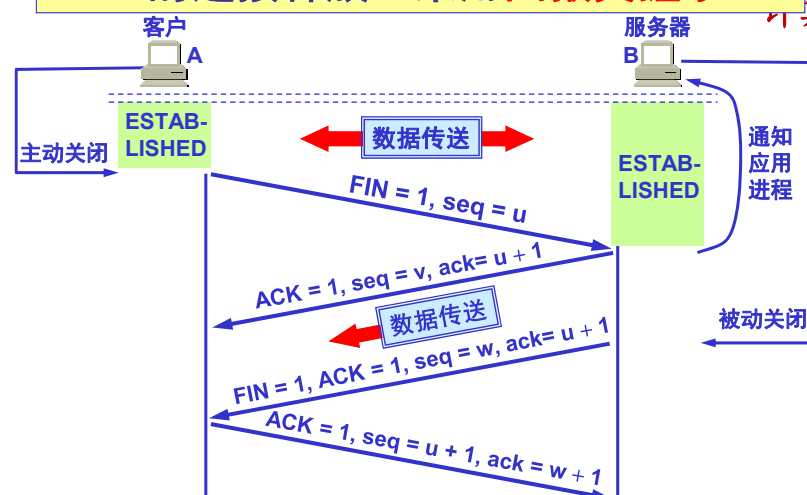
- A 收到连接释放报文段后，必须发出确认。



福州大学
FUZHOU UNIVERSITY

143

TCP 的连接释放：采用四报文握手

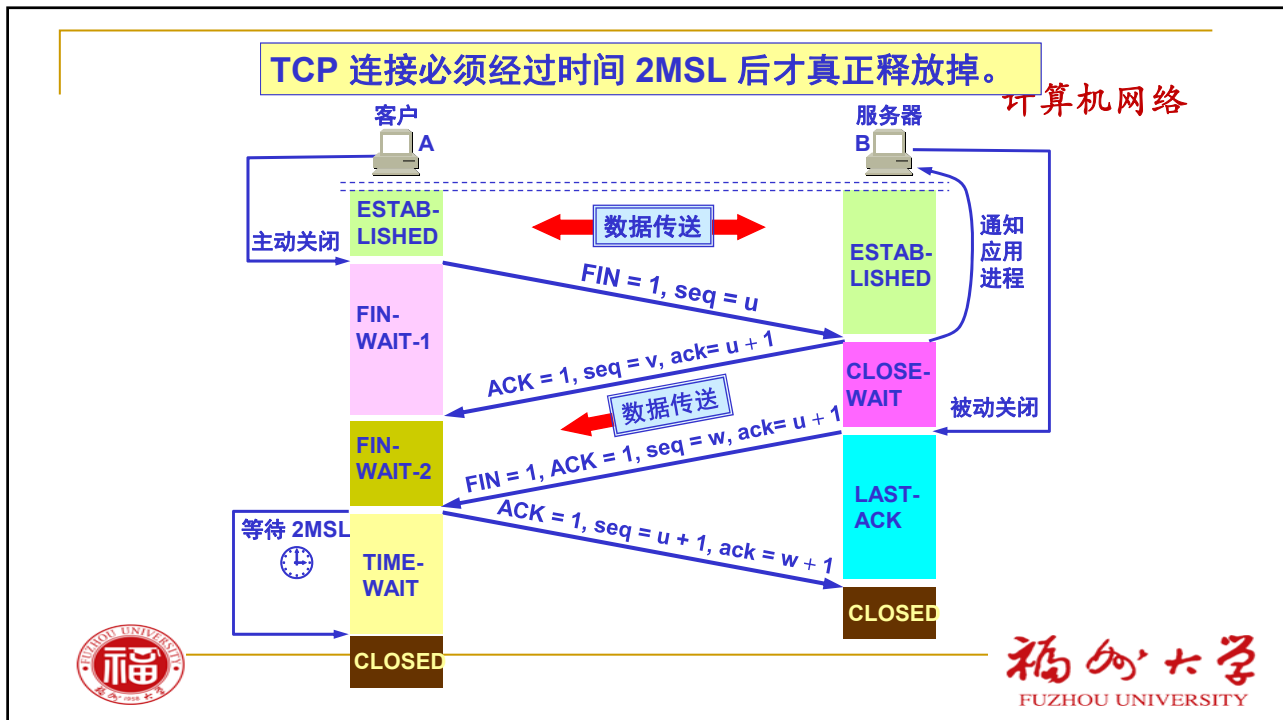


- 在确认报文段中 **ACK = 1**，确认号 **ack = w + 1**，自己的序号 **seq = u + 1**。



福州大学
FUZHOU UNIVERSITY

144



145

A 必须等待 2MSL 的时间

计算机网络

- **第一**，为了保证 A 发送的最后一个 **ACK** 报文段能够到达 B。
- **第二**，防止“已失效的连接请求报文段”出现在本连接中。A 在发送完最后一个 **ACK** 报文段后，再经过时间 **2MSL**，就可以使本连接持续的时间内所产生的所有报文段，都从网络中消失。这样就可以使下一个新的连接中不会出现这种旧的连接请求报文段。

福州大学
FUZHOU UNIVERSITY

146

3 TCP 的有限状态机

计算机网络

- TCP 的有限状态机可以更清晰地看出 TCP 连接的各种状态之间的关系。
- TCP 有限状态机的图中每一个方框都是 TCP 可能具有的状态。
- 每个方框中的大写英文字符串是 TCP 标准所使用的 TCP 连接状态名。
- 状态之间的箭头表示可能发生的状态变迁。



福州大学
FUZHOU UNIVERSITY

147

3 TCP 的有限状态机

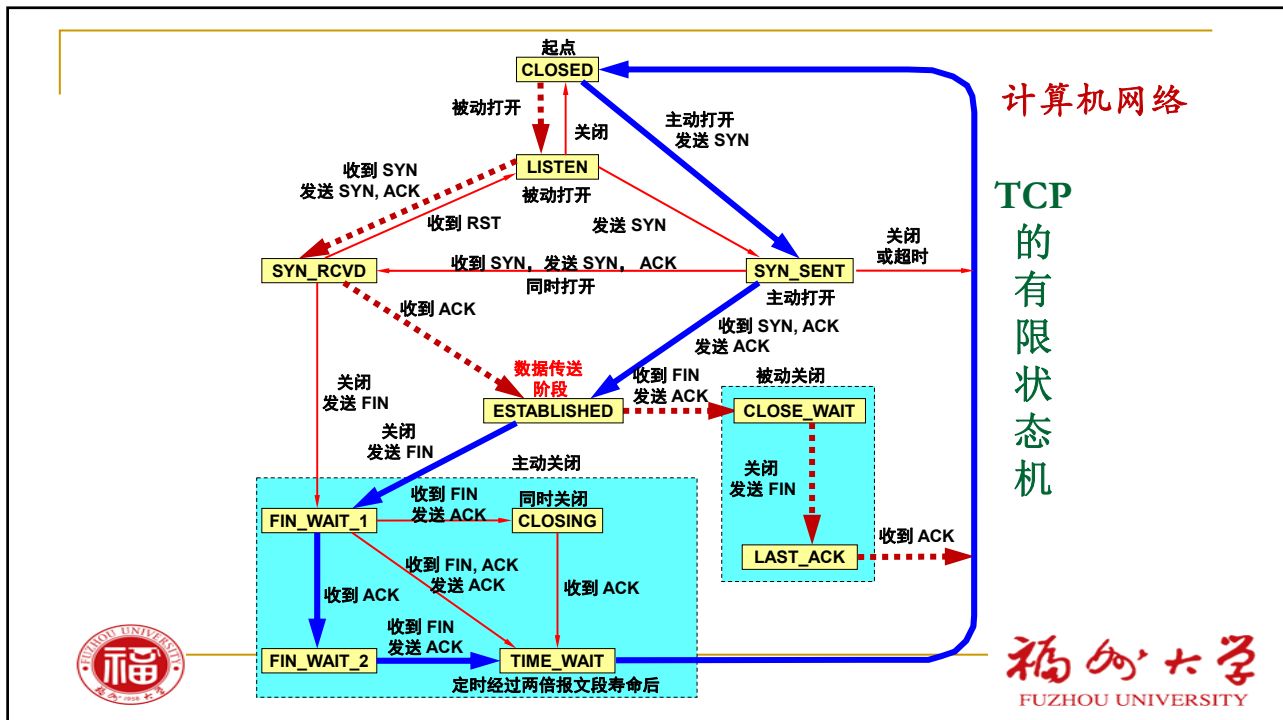
计算机网络

- 箭头旁边的字，表明引起这种变迁的原因，或表明发生状态变迁后又出现什么动作。
- 图中有三种不同的箭头。
 - 粗实线箭头表示对客户进程的正常变迁。
 - 粗虚线箭头表示对服务器进程的正常变迁。
 - 细线箭头表示异常变迁。



福州大学
FUZHOU UNIVERSITY

148



149

5.4 TCP 的拥塞控制

计算机网络

- 5.4.1 拥塞的原因与危害
- 5.4.2 拥塞控制的基本方法
- 5.4.3 TCP的拥塞控制



福州大学
FUZHOU UNIVERSITY

150

5.4 TCP 的拥塞控制

计算机网络

- 如果网络中的负载(load)，即发送到网络中的分组数量，超过了网络的容量，即网络中能处理的分组数量，那么在网络中就会**发生拥塞(congestion)**。
- 所谓**拥塞控制(congestion control)**就是防止过多的数据注入到网络中，这样可以使网络中的路由器或链路不致过载。

Σ 对资源需求 > 可用资源

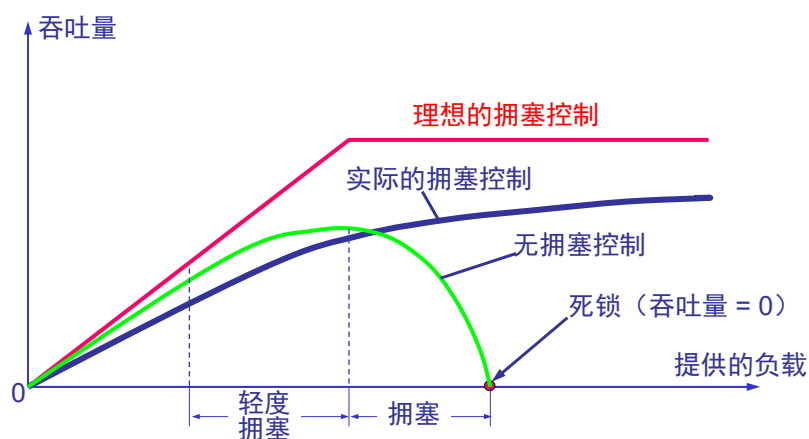


福州大学
FUZHOU UNIVERSITY

151

5.4.1 拥塞的原因与危害

计算机网络



福州大学
FUZHOU UNIVERSITY

152

拥塞控制的一般原理

计算机网络

- 实践证明，拥塞控制是很难设计的，因为它是一个**动态**的（而不是静态的）**问题**。
- 当前网络正朝着高速化的方向发展，这很容易出现缓存不够大而造成分组的丢失。**但分组的丢失是网络发生拥塞的征兆而不是原因**。
- 在许多情况下，甚至正是拥塞控制本身成为引起网络性能恶化**甚至发生死锁的原因**。这点应特别引起重视。



福州大学
FUZHOU UNIVERSITY

153

增加资源能解决拥塞吗？

计算机网络

- **不能**。这是因为网络拥塞是一个非常复杂的问题。简单地采用上述做法，在许多情况下，不但不能解决拥塞问题，而且还可能使网络的性能更坏。
- 网络拥塞往往是由许多因素引起的。例如：
 - 增大缓存，但未提高输出链路的容量和处理机的速度，排队等待时间将会大大增加，引起大量超时重传，解决不了网络拥塞；
 - 提高处理机处理的速率会将瓶颈转移到其他地方；



福州大学
FUZHOU UNIVERSITY

154

拥塞常常趋于恶化

计算机网络

- 如果一个路由器没有足够的缓存空间，它就会丢弃一些新到的分组。
- 但当分组被丢弃时，发送这一分组的源点就会重传这一分组，甚至可能还要重传多次。这样会引起更多的分组流入网络和被网络中的路由器丢弃。
- 可见拥塞引起的重传并不会缓解网络的拥塞，反而会加剧网络的拥塞。



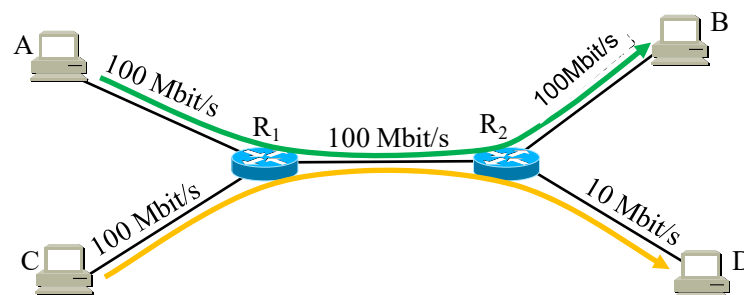
福州大学
FUZHOU UNIVERSITY

155

一个简单的网络拥塞例子

计算机网络

- 理想吞吐量为**100 Mb/s**
- 不加任何控制只能达到**60 Mb/s**
- 当分组丢失时，任何用于传输该分组的上游传输能力都被**浪费**！



福州大学
FUZHOU UNIVERSITY

156

5.4.2 拥塞控制的基本方法

计算机网络

拥塞控制与流量控制的区别

- **拥塞控制**就是防止过多的数据注入到网络中，使网络中的路由器或链路不致过载。
- **拥塞控制**所要做的都有一个前提，就是网络能够承受现有的网络负荷。
- **拥塞控制**是一个全局性的过程，涉及到所有的主机、所有的路由器，以及与降低网络传输性能有关的所有因素。



福州大学
FUZHOU UNIVERSITY

157

5.4.2 拥塞控制的基本方法

计算机网络

拥塞控制与流量控制的区别

- **流量控制**往往指点对点通信量的控制，是个端到端的问题（接收端控制发送端）。
- **流量控制**所要做的就是抑制发送端发送数据的速率，以便使接收端来得及接收。

拥塞控制和流量控制之所以常常被弄混，是因为某些拥塞控制算法是向发送端发送控制报文，并告诉发送端，网络已出现麻烦，必须放慢发送速率。这点又和流量控制是很相似的。



福州大学
FUZHOU UNIVERSITY

158

开环控制和闭环控制

计算机网络

- **开环控制**方法就是在设计网络时事先将有关发生拥塞的因素考虑周到，力求网络在工作时不产生拥塞。
- **闭环控制方法**是基于反馈环路的概念。属于闭环控制的有以下几种措施：
 - (1) 监测网络系统以便检测到拥塞在何时、何处发生。
 - (2) 将拥塞发生的信息传送到可采取行动的地方。
 - (3) 调整网络系统的运行以解决出现的问题。



福州大学
FUZHOU UNIVERSITY

159

监测网络的拥塞的指标

计算机网络

- 主要指标有：
 - 由于缺少缓存空间而被丢弃的分组的百分数；
 - 平均队列长度；
 - 超时重传的分组数；
 - 平均分组时延；
 - 分组时延的标准差，等等。
- 上述这些指标的上升都标志着拥塞的增长。



福州大学
FUZHOU UNIVERSITY

160

显式反馈和隐式反馈

计算机网络

- 根据拥塞反馈信息的形式又可以将闭环拥塞控制算法分为显式反馈算法和隐式反馈算法。
- 在显式反馈算法中，从拥塞点（即路由器）向源端提供关于网络中拥塞状态的显式反馈信息。
- 在隐式反馈算法中，源端通过对网络行为的观察（如分组丢失与往返时延）来推断网络是否发生了拥塞。
- **TCP**采用的就是隐式反馈算法。



福州大学
FUZHOU UNIVERSITY

161

5.4.3 TCP的拥塞控制

计算机网络

- **TCP** 采用**基于窗口的方法**进行拥塞控制。该方法属于闭环控制方法。
- **TCP**发送方维持一个**拥塞窗口 CWND (Congestion Window)**
 - 拥塞窗口的大小取决于网络的拥塞程度，并且动态地在变化。
 - 发送端利用**拥塞窗口**根据网络的拥塞情况调整发送的数据量。
 - 所以，发送窗口大小不仅取决于接收方公告的接收窗口，还取决于网络的拥塞状况，所以真正的发送窗口值为：

真正的发送窗口值 = $\text{Min}(\text{公告窗口值}, \text{拥塞窗口值})$



福州大学
FUZHOU UNIVERSITY

162

控制拥塞窗口的原则

计算机网络

- 只要网络没有出现拥塞，拥塞窗口就可以再增大一些，以便把更多的分组发送出去，这样就可以提高网络的利用率
- 但只要网络出现拥塞或有可能出现拥塞，就必须把拥塞窗口减小一些，以减少注入到网络中的分组数，以便缓解网络出现的拥塞。



福州大学
FUZHOU UNIVERSITY

163

拥塞的判断

计算机网络

- **重传定时器超时**
 - 现在通信线路的传输质量一般都很好，因传输出差错而丢弃分组的概率是很小的（远小于 1 %）。只要出现了超时，就可以猜想网络可能出现了拥塞。
- **收到三个相同（重复）的 ACK**
 - 个别报文段会在网络中丢失，预示可能会出现拥塞（实际未发生拥塞），因此可以尽快采取控制措施，避免拥塞。



福州大学
FUZHOU UNIVERSITY

164

TCP拥塞控制算法

计算机网络

- 四种（RFC 5681）：
 - 慢开始 (slow-start)
 - 拥塞避免 (congestion avoidance)
 - 快重传 (fast retransmit)
 - 快恢复 (fast recovery)



福州大学
FUZHOU UNIVERSITY

165

慢开始 (Slow start)

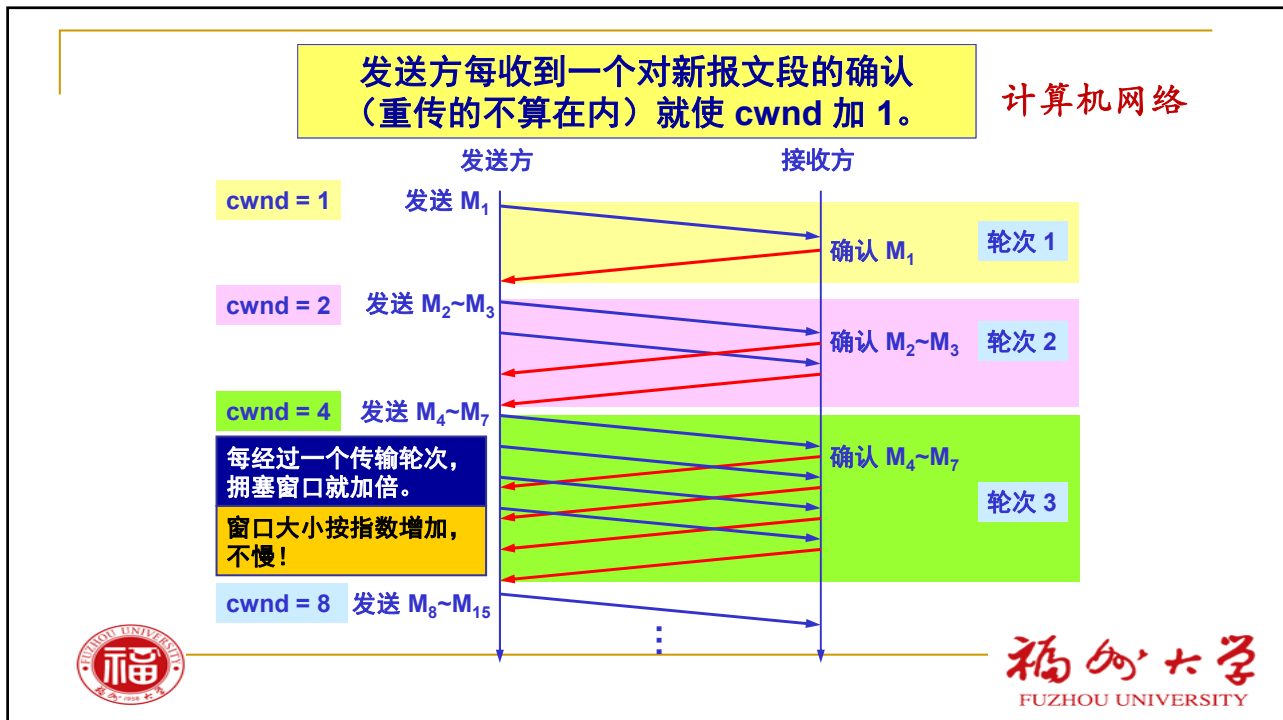
计算机网络

- 用来确定网络的负载能力。
- 算法的思路：由小到大逐渐增大拥塞窗口数值。
- 初始拥塞窗口 **cwnd** 设置：
 - 旧的规定：在刚刚开始发送报文段时，先把初始拥塞窗口 **cwnd** 设置为 1 至 2 个发送方的最大报文段 **SMSS (Sender Maximum Segment Size)** 的数值。
 - 新的 **RFC 5681** 把初始拥塞窗口 **cwnd** 设置为不超过2至4个**SMSS** 的数值。
- 慢开始门限 **ssthresh**（状态变量）：防止拥塞窗口 **cwnd** 增长过大引起网络拥塞。



福州大学
FUZHOU UNIVERSITY

166



167

传输轮次

计算机网络

- 使用慢开始算法后，每经过一个**传输轮次** (transmission round)，拥塞窗口 **cwnd** 就加倍。
- 一个传输轮次所经历的时间其实就是往返时间 **RTT**。
- “**传输轮次**” 更加强调：把拥塞窗口 **cwnd** 所允许发送的报文段都连续发送出去，并收到了对已发送的最后一个字节的确认。
- 例如，拥塞窗口 **cwnd = 4**，这时的往返时间 **RTT** 就是发送方连续发送 4 个报文段，并收到这 4 个报文段的确认，总共经历的时间。

福州大学
FUZHOU UNIVERSITY

168

设置慢开始门限状态变量 $ssthresh$ 计算机网络

- 慢开始门限 $ssthresh$ 的用法如下：
 - 当 $cwnd < ssthresh$ 时，使用慢开始算法。
 - 当 $cwnd > ssthresh$ 时，停止使用慢开始算法而改用拥塞避免算法。
 - 当 $cwnd = ssthresh$ 时，既可使用慢开始算法，也可使用拥塞避免算法。



拥塞避免算法

计算机网络

- **思路：**让拥塞窗口 $cwnd$ 缓慢地增大，即每经过一个往返时间 RTT 就把发送方的拥塞窗口 $cwnd$ 加 1，而不是加倍，使拥塞窗口 $cwnd$ 按线性规律缓慢增长。
- 因此在拥塞避免阶段就有“加法增大” (Additive Increase) 的特点。这表明在拥塞避免阶段，拥塞窗口 $cwnd$ 按线性规律缓慢增长，比慢开始算法的拥塞窗口增长速率缓慢得多。



当网络出现拥塞时

计算机网络

- 无论在慢开始阶段还是在拥塞避免阶段，只要发送方判断网络出现拥塞（**重传定时器超时**）：
 - $ssthresh = \max(cwnd/2, 2)$
 - $cwnd = 1$
 - 执行慢开始算法
- 这样做的目的就是要迅速减少主机发送到网络中的分组数，使得发生拥塞的路由器有足够时间把队列中积压的分组处理完毕

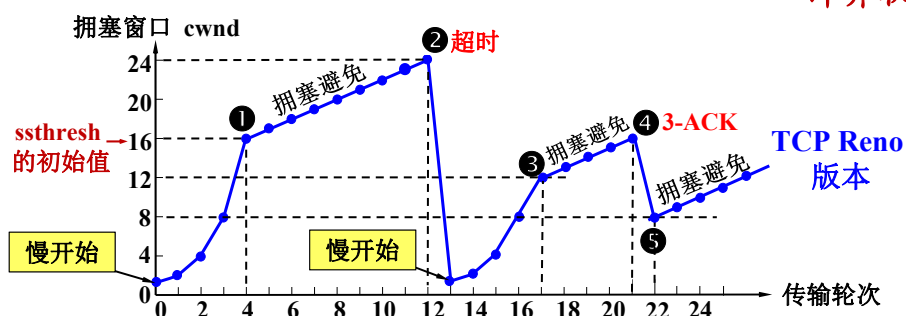


福州大学
FUZHOU UNIVERSITY

171

慢开始和拥塞避免算法的实现举例

计算机网络



当 TCP 连接进行初始化时，将拥塞窗口置为 1。图中的窗口单位不使用字节而使用报文段。

慢开始门限的初始值设置为 16 个报文段，即 $ssthresh = 16$ 。

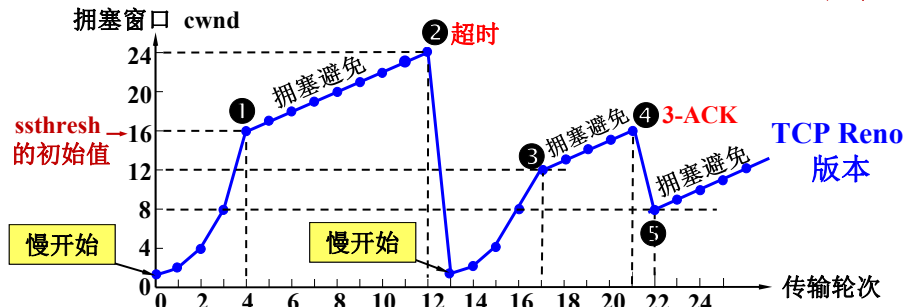


福州大学
FUZHOU UNIVERSITY

172

慢开始和拥塞避免算法的实现举例

计算机网络



发送端的发送窗口不能超过拥塞窗口 $cwnd$ 和接收端窗口 $rwnd$ 中的最小值。我们假定接收端窗口足够大，因此现在发送窗口的数值等于拥塞窗口的数值。

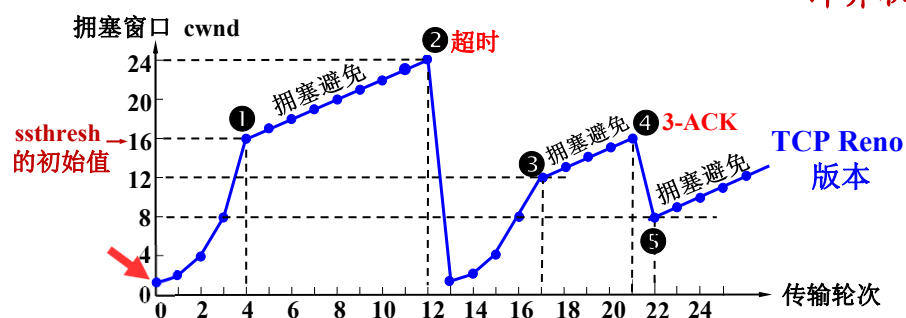


福州大学
FUZHOU UNIVERSITY

173

慢开始和拥塞避免算法的实现举例

计算机网络



在执行慢开始算法时，拥塞窗口 $cwnd=1$ ，发送第一个报文段。

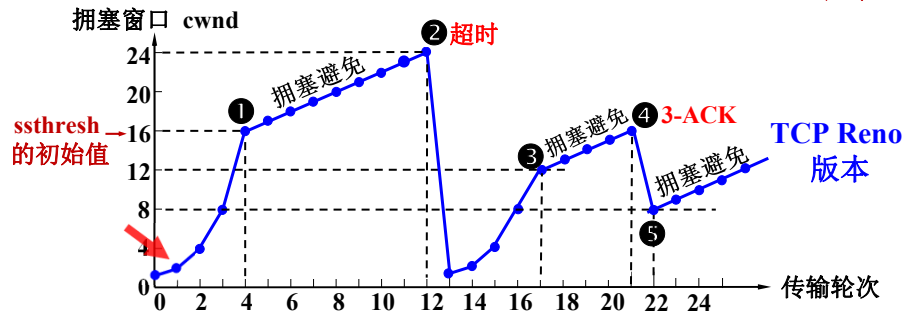


福州大学
FUZHOU UNIVERSITY

174

慢开始和拥塞避免算法的实现举例

计算机网络



发送方每收到一个对新报文段的确认 **ACK**，就把拥塞窗口值加 1，然后开始下一轮的传输（请注意，横坐标是传输轮次，不是时间）。因此拥塞窗口 **cwnd** 随着传输轮次按指数规律增长。

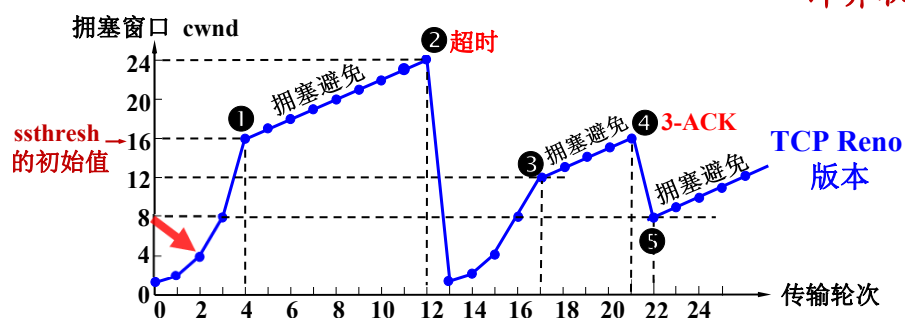


福州大学
FUZHOU UNIVERSITY

175

慢开始和拥塞避免算法的实现举例

计算机网络



发送方每收到一个对新报文段的确认 **ACK**，就把拥塞窗口值加 1，然后开始下一轮的传输（请注意，横坐标是传输轮次，不是时间）。因此拥塞窗口 **cwnd** 随着传输轮次按指数规律增长。

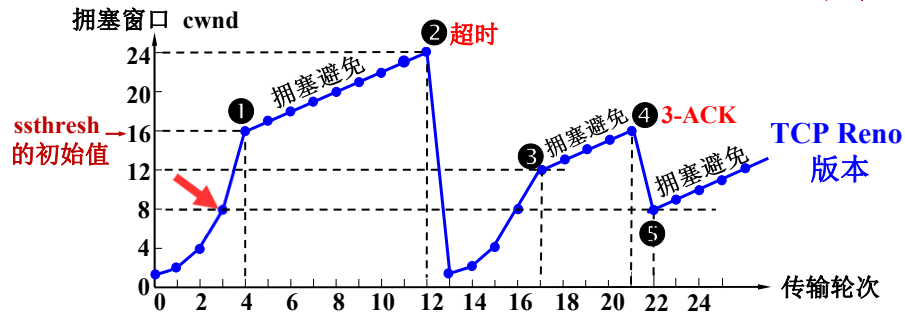


福州大学
FUZHOU UNIVERSITY

176

慢开始和拥塞避免算法的实现举例

计算机网络



发送方每收到一个新报文段的确认 ACK，就把拥塞窗口值加 1，然后开始下一轮的传输（请注意，横坐标是传输轮次，不是时间）。因此拥塞窗口 cwnd 随着传输轮次按指数规律增长。

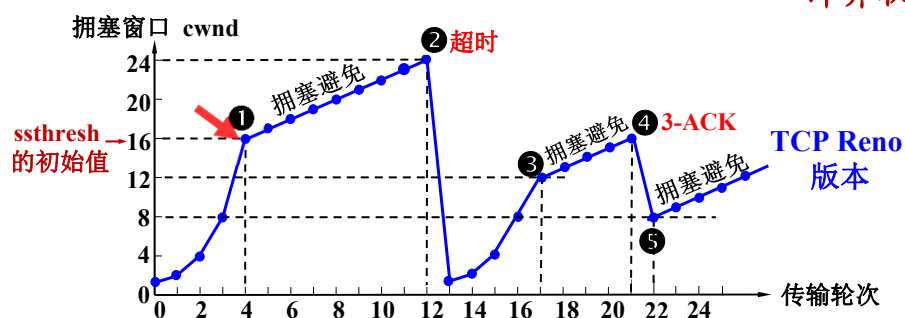


福州大学
FUZHOU UNIVERSITY

177

慢开始和拥塞避免算法的实现举例

计算机网络



当拥塞窗口 cwnd 增长到慢开始门限值 ssthresh 时（图中的点①，此时拥塞窗口 cwnd = 16），就改为执行拥塞避免算法，拥塞窗口按线性规律增长。



福州大学
FUZHOU UNIVERSITY

178

必须强调指出

计算机网络

- “拥塞避免”并非指完全能够避免了拥塞。利用以上的措施要完全避免网络拥塞还是不可能的。
- “拥塞避免”是说在拥塞避免阶段把拥塞窗口控制为按线性规律增长，使网络比较不容易出现拥塞。

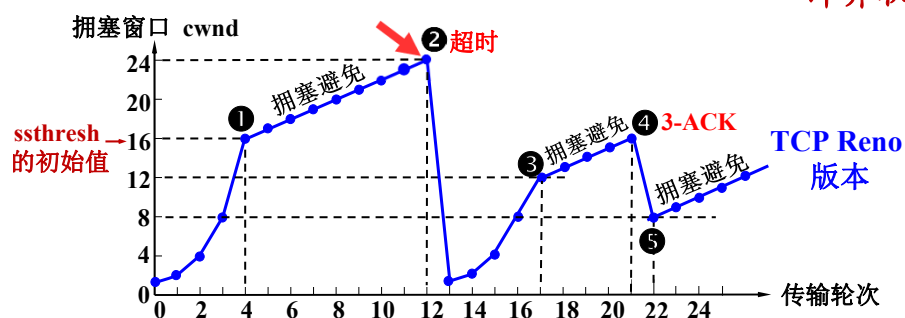


福州大学
FUZHOU UNIVERSITY

179

慢开始和拥塞避免算法的实现举例

计算机网络



当拥塞窗口 $cwnd = 24$ 时，网络出现了超时（图中的点②），发送方判断为网络拥塞。于是调整门限值 $sssthresh = cwnd / 2 = 12$ ，同时设置拥塞窗口 $cwnd = 1$ ，进入慢开始阶段。

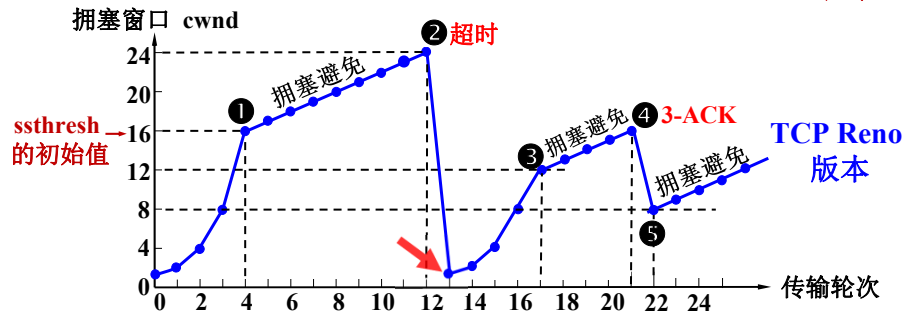


福州大学
FUZHOU UNIVERSITY

180

慢开始和拥塞避免算法的实现举例

计算机网络



当拥塞窗口 $cwnd = 24$ 时，网络出现了**超时**（图中的点②），发送方判断为网络拥塞。于是**调整门限值** $ssthresh = cwnd / 2 = 12$ ，同时设置拥塞窗口 $cwnd = 1$ ，进入**慢开始**阶段。

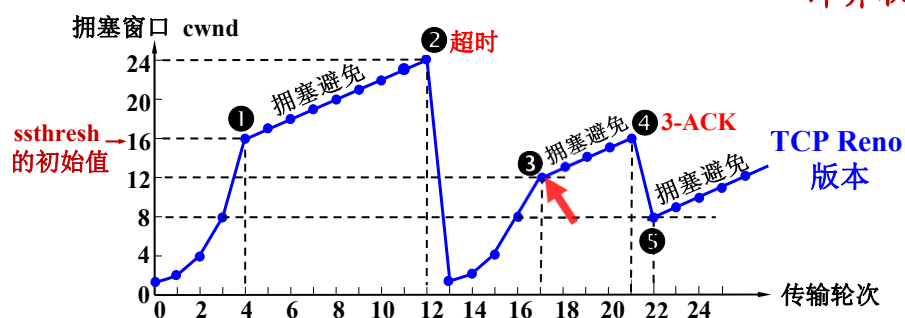


福州大学
FUZHOU UNIVERSITY

181

慢开始和拥塞避免算法的实现举例

计算机网络



按照慢开始算法，发送方每收到一个对新报文段的确认ACK，就把拥塞窗口值加1。
当拥塞窗口 $cwnd = ssthresh = 12$ 时（图中的点③，这是新的ssthresh值），改为执行**拥塞避免**算法，拥塞窗口**按线性规律增大**。

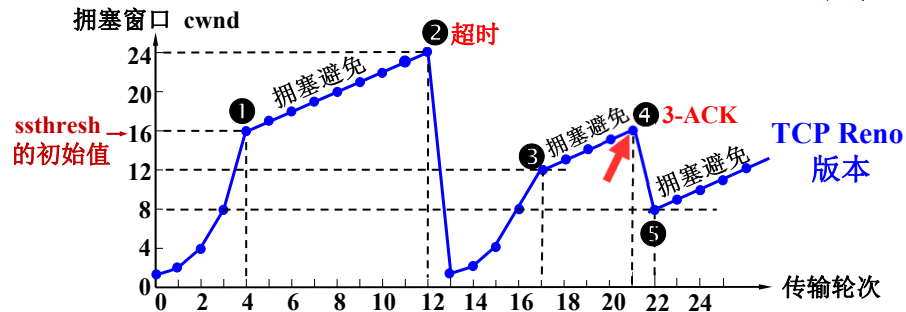


福州大学
FUZHOU UNIVERSITY

182

慢开始和拥塞避免算法的实现举例

计算机网络



当拥塞窗口 $cwnd = 16$ 时（图中的点④），出现了一个新的情况，就是发送方一连收到 3 个对同一个报文段的重复确认（图中记为 3-ACK）。发送方改为执行快重传和快恢复算法。

福州大学
FUZHOU UNIVERSITY

183

快重传算法

计算机网络

- 采用快重传FR (Fast Retransmission) 算法可以让发送方尽早知道发生了个别报文段的丢失。
- 快重传 算法首先要求接收方不要等待自己发送数据时才进行捎带确认，而是要立即发送确认，即使收到了失序的报文段也要立即发出对已收到的报文段的重复确认。

福州大学
FUZHOU UNIVERSITY

184

快重传算法

计算机网络

- 发送方只要一连收到三个重复确认，就知道接收方确实没有收到报文段，因而应当立即进行重传（即“快重传”），这样就不会出现超时，发送方也不就会误认为出现了网络拥塞。
- 使用快重传可以使整个网络的吞吐量提高约**20%**。

不难看出，快重传并非取消重传计时器，而是在某些情况下可**更早地重传**丢失的报文段。

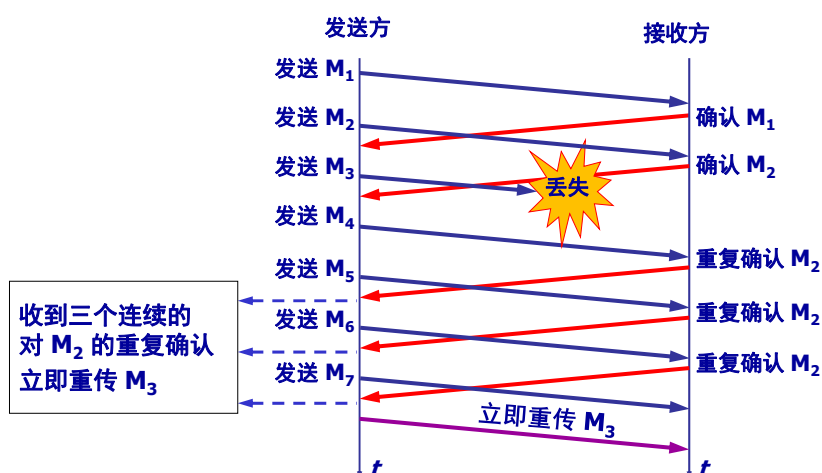


福州大学
FUZHOU UNIVERSITY

185

快重传举例

计算机网络



福州大学
FUZHOU UNIVERSITY

186

快恢复算法

计算机网络

- 当发送端收到连续三个重复的确认时，由于发送方现在认为网络很可能没有发生拥塞，因此现在**不执行慢开始算法**，而是执行**快恢复算法 FR (Fast Recovery)** 算法：

- (1) 慢开始门限 $ssthresh = \text{当前拥塞窗口 } cwnd / 2$ ；
- (2) 新拥塞窗口 $cwnd = \text{慢开始门限 } ssthresh$ ；
- (3) 开始执行拥塞避免算法，使拥塞窗口缓慢地线性增大。

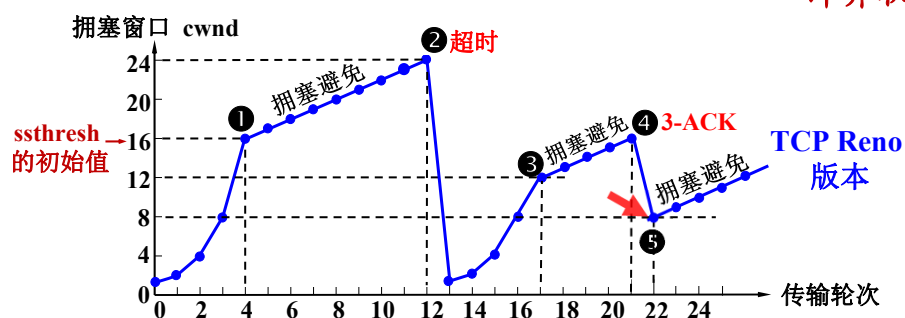


福州大学
FUZHOU UNIVERSITY

187

慢开始和拥塞避免算法的实现举例

计算机网络



因此，在图的点④，发送方知道现在只是丢失了个别的报文段。于是**不启动慢开始，而是执行快恢复算法**。这时，发送方调整门限值 $ssthresh = cwnd / 2 = 8$ ，同时设置拥塞窗口 $cwnd = ssthresh = 8$ （见图中的点⑤），并开始执行拥塞避免算法。



福州大学
FUZHOU UNIVERSITY

188

加法增大，乘法减小 (AIMD)

计算机网络

- 可以看出，在拥塞避免阶段，拥塞窗口是按照线性规律增大的。这常称为“**加法增大**” **AI (Additive Increase)**。
- 当出现超时或3个重复的确认时，就要把门限值设置为当前拥塞窗口值的一半，并大大减小拥塞窗口的数值。这常称为“**乘法减小**” **MD (Multiplicative Decrease)**。
- 二者合在一起就是所谓的 **AIMD** 算法。

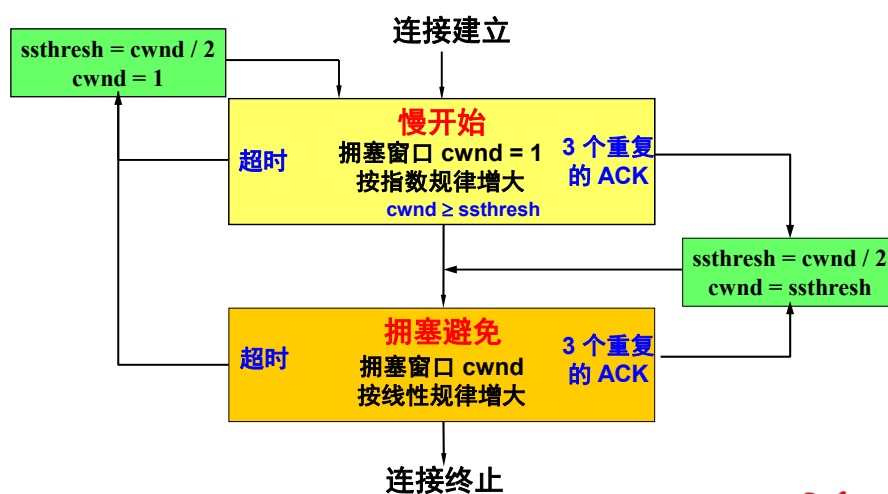


福州大学
FUZHOU UNIVERSITY

189

TCP拥塞控制流程图

计算机网络



福州大学
FUZHOU UNIVERSITY

190

发送窗口的上限值

计算机网络

- 发送方的发送窗口的上限值应当取为接收方窗口 **rwnd** 和拥塞窗口 **cwnd** 这两个变量中较小的一个，即应按以下公式确定：

$$\text{发送窗口的上限值} = \text{Min} [\text{rwnd}, \text{cwnd}]$$

- 当 $\text{rwnd} < \text{cwnd}$ 时，是接收方的接收能力限制发送窗口的最大值。
- 当 $\text{cwnd} < \text{rwnd}$ 时，则是网络的拥塞限制发送窗口的最大值。

也就是说，**rwnd** 和 **cwnd** 中数值较小的一个，控制了发送方发送数据的速率。



福州大学
FUZHOU UNIVERSITY

191

主动队列管理 AQM

计算机网络

- **TCP** 拥塞控制和网络层采取的策略有密切联系。
 - 若路由器对某些分组的处理时间特别长，就可能引起发送方 **TCP** 超时，对这些报文段进行重传。
 - 重传会使 **TCP** 连接的发送端认为在网络中发生了拥塞。于是在 **TCP** 的发送端就采取了拥塞控制措施，但实际上网络并没有发生拥塞。
- 网络层的策略对 **TCP** 拥塞控制影响最大的就是路由器的分组丢弃策略。



福州大学
FUZHOU UNIVERSITY

192

“先进先出” FIFO 处理规则

计算机网络

- 路由器的队列通常都是按照“先进先出” FIFO (First In First Out) 的规则处理到来的分组。
- 当队列已满时，以后再到达的所有分组（如果能够继续排队，这些分组都将排在队列的尾部）将都被丢弃。这就叫做**尾部丢弃策略 (tail-drop policy)**。
- 路由器的尾部丢弃往往会导致一连串分组的丢失，这就使发送方出现超时重传，使 TCP 进入拥塞控制的慢开始状态，结果使 TCP 连接的发送方突然把数据的发送速率降低到很小的数值。



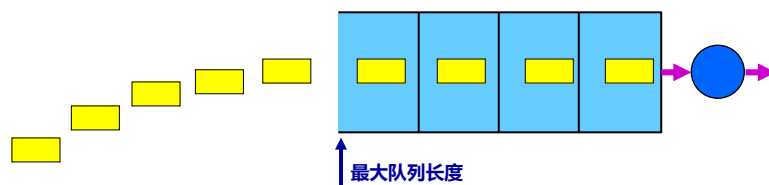
福州大学
FUZHOU UNIVERSITY

193

先进先出（FIFO）处理规则与尾部丢弃策略

计算机网络

- 在最简单的情况下，路由器队列通常采用
- 先进先出 (FIFO)** 处理规则与**尾部丢弃策略 (tail-drop policy)**。
- 当队列已满时，以后到达的所有分组将都被丢弃。



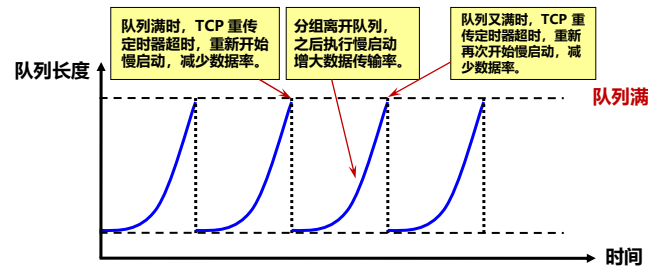
福州大学
FUZHOU UNIVERSITY

194

先进先出（FIFO）处理规则与尾部丢弃策略

计算机网络

- 分组丢弃使发送方出现超时重传，使 TCP 连接进入慢开始状态。



福州大学
FUZHOU UNIVERSITY

195

严重问题：全局同步

计算机网络

- 更为严重的是，在网络中通常有很多的 TCP 连接，这些连接中的报文段通常是复用在网络层的 IP 数据报中传送的。
- 在这种情况下，若发生了路由器中的尾部丢弃，就可能会同时影响到很多条 TCP 连接，结果使这许多 TCP 连接在同一时间突然都进入到慢开始状态。这在 TCP 的术语中称为**全局同步 (global synchronization)**。
- 全局同步使得全网的通信量突然下降了很多，而在网络恢复正常后，其通信量又突然增大很多。



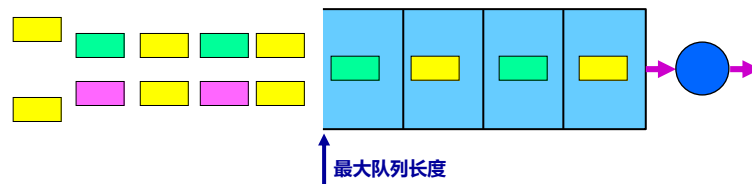
福州大学
FUZHOU UNIVERSITY

196

严重问题：全局同步

计算机网络

- 若路由器进行了尾部丢弃，
- 所有到达的分组都被丢弃，不论它们属于哪个 **TCP** 连接。



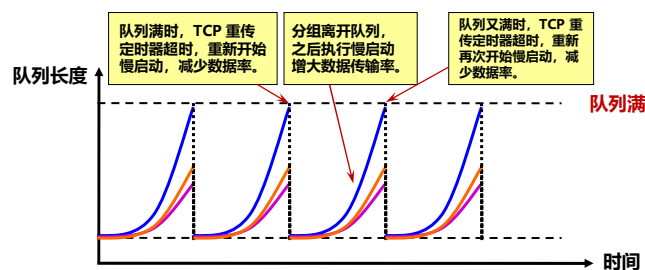
福州大学
FUZHOU UNIVERSITY

197

严重问题：全局同步

计算机网络

- 分组丢弃使发送方出现超时重传，使多个 **TCP** 连接同时进入慢开始状态，
- 发生全局同步 (global synchronization)。



福州大学
FUZHOU UNIVERSITY

198

主动队列管理 AQM

计算机网络

- 1998年提出了主动队列管理 AQM (Active Queue Management)。
- 所谓“主动”就是不要等到路由器的队列长度已经达到最大值时才不得不丢弃后面到达的分组。这样就太被动了。应当在队列长度达到某个值得警惕的数值时（即当网络拥塞有了某些拥塞征兆时），就主动丢弃到达的分组。
- AQM 可以有不同实现方法，其中曾流行多年的就是随机早期检测 RED (Random Early Detection)。



福州大学
FUZHOU UNIVERSITY

199

随机早期检测 RED

计算机网络

- 使路由器的队列维持两个参数：
 - 队列长度最小门限 TH_{min} 和最大门限 TH_{max} 。
- RED 对每一个到达的分组都先计算平均队列长度 L_{AV} 。
 - (1) 若平均队列长度小于最小门限 TH_{min} ，则将新到达的分组放入队列进行排队。
 - (2) 若平均队列长度超过最大门限 TH_{max} ，则将新到达的分组丢弃。
 - (3) 若平均队列长度在最小门限 TH_{min} 和最大门限 TH_{max} 之间，则按照某一概率 p 将新到达的分组丢弃。



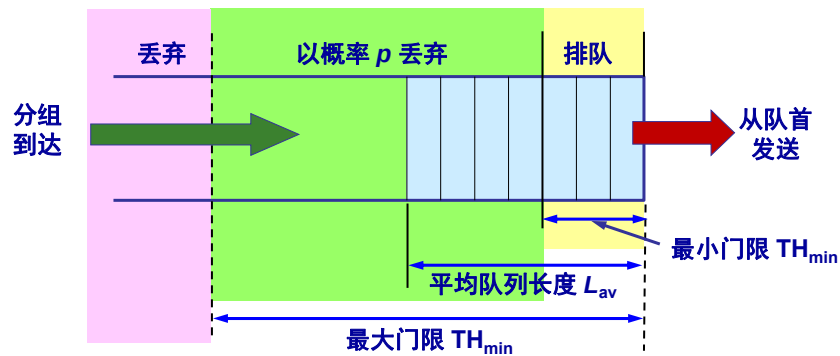
福州大学
FUZHOU UNIVERSITY

200

随机早期检测 RED

计算机网络

RED 将路由器的到达队列划分成为三个区域：



RED 对每一个到达的分组都先计算平均队列长度 L_{AV} 。



福州大学
FUZHOU UNIVERSITY

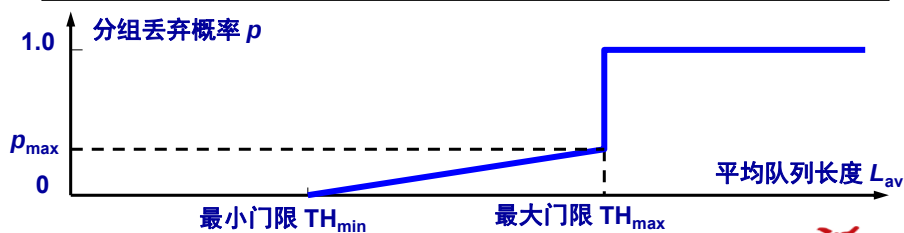
201

丢弃概率 p 与 TH_{min} 和 TH_{max} 的关系

计算机网络

- 当 $L_{AV} < TH_{min}$ 时，丢弃概率 $p = 0$ 。
- 当 $L_{AV} > TH_{max}$ 时，丢弃概率 $p = 1$ 。
- 当 $TH_{min} < L_{AV} < TH_{max}$ 时， $0 < p < 1$ 。

在 RED 的操作中，最难处理的就是丢弃概率 p 的选择，因为 p 并不是个常数。例如，按线性规律变化，从 0 变到 p_{max} 。



福州大学
FUZHOU UNIVERSITY

202

随机早期检测 RED

计算机网络

- 多年的实践证明，**RED** 的使用效果并不太理想。
- **2015**年公布的 **RFC 7567** 已经把 **RFC 2309**列为陈旧的，并且不再推荐使用 **RED**。
- 对路由器进行主动队列管理 **AQM** 仍是必要的。
- **AQM** 实际上就是对路由器中的分组排队进行智能管理，而不是简单地把队列的尾部丢弃。
- 现在已经有几种不同的算法来代替旧的 **RED**，但都还在实验阶段。



福州大学
FUZHOU UNIVERSITY

203

课后作业

计算机网络

- **5-11、5-12、5-16、5-18、5-21、5-23、5-24、5-29、5-30、5-32、5-33、5-34、5-37、5-38、5-39、5-42、5-43、5-47、5-68**



福州大学
FUZHOU UNIVERSITY

204